



TRABAJO FINAL DE CARRERA

Automatización de Gestión Empresarial con n8n: Asistente Inteligente para Taller Mecánico vía Telegram

Línea de Investigación: Workflows Aplicados a la Gestión Empresarial y Marketing

Tecnicatura Universitaria en Programación

Universidad Tecnológica Nacional

Mendoza, 2026

Profesor: Cortez, Alberto Alejandro

Estudiantes: Palmero, Manuel; Rojas, Uriel

Resumen.....	4
I. Introducción.....	5
1.1. Contexto y Justificación.....	5
1.2. Problemática Identificada.....	6
1.2.1. Hipótesis de Trabajo.....	6
1.3. Objetivos.....	7
1.3.1. Objetivo General.....	7
1.3.2. Objetivos Específicos.....	7
II. Marco Teórico.....	9
2.1. Automatización de Procesos de Negocio (BPA/RPA).....	9
2.2. Plataformas Low-Code y n8n.....	9
2.3. Agentes Conversacionales basados en LLM.....	10
2.4. Integración del Framework LangChain.....	10
2.5. Modelos de Lenguaje de Gran Escala: Mistral AI.....	11
2.6. Persistencia de Datos en Hojas de Cálculo Conectadas.....	11
2.7. Telegram Bot API como Interfaz Conversacional.....	12
2.8. Antecedentes y Estado del Arte.....	12
2.8.1. Metodología de búsqueda.....	12
2.8.2. Automatización conversacional y agentes con uso de herramientas.....	12
2.8.3. Digitalización de PyMEs en el sector automotriz y de servicios.....	12
2.8.4. Brecha identificada y posicionamiento del trabajo.....	13
III. Marco Metodológico.....	14
3.1. Tipo y Enfoque de Investigación.....	14
3.2. Diseño de Investigación: Estudio de Caso.....	14
3.3. Caracterización del Caso de Estudio.....	14
3.4. Instrumentos de Relevamiento.....	15
3.5. Procedimiento de Desarrollo Iterativo.....	15
3.6. Técnicas de Análisis y Validación.....	16
IV. Diseño de la Solución.....	18
4.1. Arquitectura General del Workflow.....	18
4.2. Lógica de Enrutamiento.....	18
4.3. Agente de Servicio al Cliente (Público).....	19
4.4. Agente Administrativo (Privado).....	20
4.5. Mecanismo de Seguridad y Confirmación.....	20
4.6. Mecanismo de Paginación de Listados.....	21
4.7. Reglas de Negocio Incorporadas.....	21
4.8. Estructura de la Base de Datos.....	22
4.9. Limitaciones Técnicas Reconocidas.....	23
V. Implementación.....	24

5.1. Configuración del Bot de Telegram.....	24
5.2. Configuración de los Agentes de IA.....	24
5.3. Integración con Google Sheets.....	24
5.4. Configuración del Modelo Mistral.....	25
5.5. Subworkflow de Consulta de Disponibilidad.....	25
5.6. Mecanismo de Confirmación y Paginación por Botones.....	25
5.7. Mecanismo de Respuesta.....	25
VI. Resultados y Validación Funcional.....	26
6.1. Diseño Experimental de las Pruebas.....	26
6.2. Resultados de las Pruebas Unitarias.....	26
6.3. Resultados de las Pruebas de Integración.....	27
6.4. Resultados de las Pruebas de Enrutamiento y Seguridad.....	27
6.5. Discusión de Resultados.....	28
VII. Conclusiones.....	29
7.1. Cumplimiento de Objetivos.....	29
7.2. Aportes del Trabajo.....	30
7.3. Limitaciones del Estudio.....	30
7.4. Líneas de Trabajo Futuro.....	31
VIII. Referencias Bibliográficas.....	32
Documentación técnica consultada.....	33
IX. Anexos.....	34
Anexo 8.1. Prompts de Sistema Completos.....	34
8.1.1. Prompt del Agente Público (Servicio Técnico).....	34
8.1.2. Prompt del Agente Privado (Panel Administrativo).....	35
Anexo 8.2. Diagrama de Flujo del Workflow Principal.....	36
Anexo 8.3. Modelo de Datos Lógico.....	37
Anexo 8.4. Guion de Entrevista al Caso de Estudio.....	37
Anexo 8.5. Capturas de Pantalla de Pruebas Funcionales.....	38

Resumen

El presente Trabajo Final de Carrera desarrolla, implementa y valida un sistema de automatización para la gestión operativa de un taller mecánico de pequeña escala, utilizando la plataforma de orquestación de workflows de código abierto n8n integrada con la aplicación de mensajería Telegram como canal conversacional. La arquitectura se fundamenta en la concepción de dos agentes de inteligencia artificial diferenciados bajo el principio de privilegio mínimo: un agente de atención al cliente, orientado al registro, la modificación y la cancelación de turnos y a la consulta de inventario, y un agente administrativo, restringido al personal interno, para la gestión completa de la base de datos del taller. Como motor de razonamiento se emplea el modelo Mistral Cloud, integrado mediante el framework LangChain en su implementación nativa de n8n. La persistencia de datos se gestiona en Google Sheets, estructurada en tres hojas vinculadas (stock, clientes y turnos). Respecto de la versión inicial del prototipo, el sistema incorpora en su estado actual un conjunto de reglas de negocio adicionales: un límite máximo de tres turnos pendientes por cliente, la verificación obligatoria de disponibilidad horaria antes de solicitar cualquier dato personal, el respeto estricto del horario comercial del taller, la posibilidad de cancelar y reprogramar turnos ya existentes, un mecanismo de confirmación mediante botones interactivos de Telegram para toda operación administrativa sensible, y un sistema de paginación con navegación por botones para listados extensos de clientes, turnos o repuestos.

Metodológicamente se adoptó un diseño de estudio de caso instrumental único, aplicado a un taller mecánico del Gran Mendoza caracterizado mediante entrevista semiestructurada y observación no participante. El desarrollo siguió un modelo iterativo e incremental, y el sistema fue sometido a un protocolo de validación funcional compuesto por pruebas unitarias, de integración, de enrutamiento y de seguridad.

Los hallazgos confirman la viabilidad técnica de la solución en su condición de prototipo de investigación, validado en condiciones controladas. El mecanismo de autenticación administrativa se implementa mediante la validación del identificador de usuario de Telegram (User ID) contra una lista de identificadores autorizados definida en el propio workflow, con independencia total del contenido textual del mensaje recibido. El trabajo aporta un modelo replicable de adopción de tecnologías low-code combinadas con agentes de lenguaje para PyMEs del sector automotriz argentino, y traza líneas futuras orientadas a la persistencia conversacional, la migración de la base de datos y la integración con calendarios externos. Su transformación en un sistema apto para despliegue productivo requiere, adicionalmente, un análisis de conformidad con la Ley 25.326 de Protección de Datos Personales.

Palabras clave: automatización de procesos de negocio, low-code, agentes de inteligencia artificial, LangChain, PyMEs, Telegram Bot, n8n.

I. Introducción

1.1. Contexto y Justificación

La transformación digital se ha consolidado como un eje estratégico para la competitividad empresarial contemporánea, y la automatización de procesos de negocio (Business Process Automation, BPA) ocupa dentro de ella un lugar central (van der Aalst et al., 2018). Para las pequeñas y medianas empresas (PyMEs), la adopción tecnológica enfrenta restricciones presupuestarias y de capital humano especializado que tornan particularmente atractivas las soluciones de código abierto y de bajo costo de implementación (Eller et al., 2020; Sahay et al., 2020).

En el sector automotriz, los talleres mecánicos de pequeña escala presentan procesos operativos altamente susceptibles de automatización. La gestión de turnos, el control de inventario de repuestos y la comunicación con clientes suelen sostenerse mediante registros manuales o herramientas ofimáticas básicas, lo que genera ineficiencias documentadas en la literatura sobre digitalización de PyMEs de servicios. (Eller et al., 2020).

La plataforma n8n se inscribe en la categoría de herramientas low-code de orquestación de workflows. Su carácter de código abierto y la posibilidad de despliegue autoalojado proveen control sobre la infraestructura y la privacidad de los datos, atributos relevantes para entornos empresariales con datos sensibles de clientes. La selección de Telegram como interfaz se fundamenta en la robustez y gratuidad de su API de bots, mientras que la incorporación de un modelo de lenguaje de gran escala como Mistral permite ofrecer una interfaz conversacional en lenguaje natural, eliminando la curva de aprendizaje asociada a aplicaciones tradicionales.

La selección de Telegram como interfaz se fundamenta en la robustez y gratuidad de su API de bots, así como en su penetración en el mercado argentino. Por su parte, la incorporación de un modelo de lenguaje de gran escala (LLM) como Mistral permite ofrecer una interfaz conversacional en lenguaje natural, eliminando la curva de aprendizaje asociada a aplicaciones tradicionales y alineándose con la tendencia de adopción de agentes conversacionales en servicios al cliente (Adamopoulou & Moussiades, 2020).

1.2. Problemática Identificada

La gestión operativa en talleres mecánicos de pequeña escala presenta deficiencias estructurales recurrentes. Los principales problemas detectados se sintetizan a continuación.

La gestión de turnos resulta ineficiente cuando se sostiene mediante agendas físicas, planillas de cálculo simples o llamadas telefónicas, lo que produce pérdida de oportunidades fuera del horario comercial, dificultades para el seguimiento centralizado de la disponibilidad y solapamientos involuntarios.

El control de inventario rara vez se actualiza en tiempo real, lo que obliga al personal a realizar verificaciones manuales y favorece la aparición de faltantes críticos al momento de una reparación. La gestión de clientes adolece de dispersión de la información, dificultando la consulta de historiales y la personalización de la atención. Finalmente, la carga operativa se concentra desproporcionadamente sobre el mecánico o encargado, quien debe atender simultáneamente el teléfono, agendar turnos, consultar stock y gestionar facturación, lo que desvía su atención de las tareas técnicas y reduce la productividad general del taller.

Un problema adicional, no contemplado en versiones preliminares del sistema, es la ausencia de mecanismos de contención frente al uso desmedido del canal conversacional: sin un límite explícito, un mismo cliente podría acumular una cantidad indefinida de turnos pendientes, saturando la agenda real del taller. Asimismo, la falta de una vía para cancelar o reprogramar turnos desde el propio canal conversacional obligaba, en el diseño inicial, a resolver estos casos por fuera del sistema.

La solución propuesta busca mitigar estos problemas mediante la centralización de datos, la automatización basada en reglas explícitas y la incorporación de capacidades conversacionales de inteligencia artificial para gestionar la interacción inicial con el cliente.

1.2.1. Hipótesis de Trabajo

Dado el carácter aplicado y orientado al desarrollo tecnológico del presente trabajo, no se formula una hipótesis verificable en el sentido estricto de la investigación empírica cuantitativa. En su lugar, se plantean los siguientes supuestos de trabajo que guían las decisiones de diseño e implementación.

Primero, la integración de n8n con un modelo de lenguaje de gran escala, mediante el framework LangChain y su exposición a través de la API de Telegram Bot, permite construir un sistema de automatización conversacional técnicamente viable para la gestión operativa de un taller mecánico PyME, utilizando exclusivamente herramientas de código abierto o gratuitas.

Segundo, la separación de privilegios entre un agente público y un agente administrativo, combinada con la validación del identificador de usuario de Telegram contra una lista de identificadores autorizados, constituye un mecanismo de control de acceso

funcional para el contexto de un prototipo de investigación, con independencia del contenido del mensaje enviado.

Tercero, la incorporación de reglas de negocio explícitas (límite de turnos por cliente, verificación de disponibilidad previa, confirmación obligatoria de operaciones sensibles) reduce el margen de error operativo del sistema sin requerir intervención humana adicional.

La validación funcional del sistema mediante pruebas unitarias, de integración y de enrutamiento constituye el mecanismo de verificación de ambos supuestos.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar y validar un workflow operativo dentro de la plataforma n8n que automatice la gestión de turnos, la administración de la base de clientes y el control de stock de repuestos en un taller mecánico, accesible mediante un bot de Telegram con dos niveles de acceso diferenciados.

1.3.2. Objetivos Específicos

1. Diseñar e implementar un agente de inteligencia artificial de atención al cliente capaz de interpretar peticiones en lenguaje natural para registrar turnos, incorporar nuevos clientes y consultar la disponibilidad de repuestos.
2. Implementar un agente de gestión administrativa con acceso restringido que permita consultar el inventario completo, actualizar el stock y acceder a la lista consolidada de turnos y clientes.
3. Establecer la integración entre n8n y Google Sheets como repositorio centralizado de datos, garantizando la integridad de la información mediante autenticación OAuth2.
4. Configurar el modelo Mistral Cloud como motor de razonamiento de ambos agentes mediante los nodos nativos de n8n para LangChain.
5. Desarrollar una lógica de enrutamiento que diferencie los canales de acceso público y privado mediante la validación del identificador de usuario de Telegram (User ID) contra una lista de identificadores de administradores autorizados, almacenando el resultado en una variable booleana denominada "isAdmin" que determina el flujo de ejecución a través de un nodo Switch, con independencia del contenido del mensaje.

6. Incorporar reglas de negocio que limiten la cantidad de turnos activos por cliente, verifiquen la disponibilidad horaria antes de solicitar datos personales y respeten el horario comercial declarado del taller.
7. Implementar mecanismos de confirmación explícita mediante botones interactivos para toda operación administrativa que modifique datos, y un sistema de paginación por botones para la consulta de listados extensos.
8. Validar el sistema mediante un protocolo de pruebas funcionales unitarias, de integración y de seguridad, documentando los resultados con evidencia empírica.

II. Marco Teórico

2.1. Automatización de Procesos de Negocio (BPA/RPA)

La automatización de procesos de negocio constituye un campo consolidado de la disciplina de los sistemas de información, definido como la aplicación de tecnología para ejecutar tareas recurrentes con mínima intervención humana (van der Aalst et al., 2018). Dentro de este campo, la automatización robótica de procesos (Robotic Process Automation, RPA) ha emergido como una subdisciplina enfocada en la imitación de interacciones humanas con sistemas de software mediante agentes automatizados (Syed et al., 2020).

La distinción entre BPA y RPA es relevante para situar el presente trabajo. Mientras la RPA opera tradicionalmente sobre interfaces gráficas de aplicaciones existentes, la BPA orquesta servicios mediante interfaces de programación de aplicaciones (API), lo que permite mayor robustez, mantenibilidad y escalabilidad (Hofmann et al., 2020). El sistema desarrollado en este trabajo se inscribe plenamente en el paradigma de la BPA orientada a APIs.

Estudios recientes sobre adopción de BPA en PyMEs señalan que las principales barreras no son tecnológicas sino organizacionales, vinculadas a la falta de capital humano especializado y a la dificultad para identificar procesos automatizables (Eller et al., 2020). En este contexto, las plataformas low-code se han posicionado como mediadoras entre la complejidad técnica y la realidad operativa de las pequeñas organizaciones (Sahay et al., 2020).

2.2. Plataformas Low-Code y n8n

Las plataformas low-code permiten construir aplicaciones y workflows mediante interfaces visuales, reduciendo significativamente la cantidad de código manual requerido. Dentro del ecosistema de orquestación de workflows, las alternativas de mayor difusión incluyen Zapier, Make, Microsoft Power Automate, Apache Airflow y n8n. La selección de n8n para este trabajo se fundamenta en una comparación deliberada entre alternativas: Zapier y Make ofrecen modelos comerciales basados en suscripción con limitaciones por volumen de ejecuciones, lo que resulta inapropiado para una PyME con presupuesto restringido; Apache Airflow, si bien es de código abierto, está orientado a pipelines de datos y requiere conocimientos avanzados de programación; Power Automate presenta integración profunda con el ecosistema Microsoft pero costos asociados a las licencias. n8n, en cambio, ofrece un

modelo de código abierto con licencia fair-code que permite el despliegue autoalojado sin costos por ejecución, soporte nativo para integración con modelos de lenguaje mediante LangChain, y una arquitectura modular basada en nodos, incluyendo nodos de código personalizado (JavaScript) que permiten implementar lógica de negocio específica, como la utilizada en este trabajo para el enrutamiento, la paginación y el manejo de confirmaciones.

La arquitectura de n8n se sustenta en el concepto de nodo, entendido como unidad funcional que ejecuta una acción específica (envío de un mensaje, consulta a una API, escritura en una base de datos) o que actúa como disparador (trigger) iniciando la ejecución del workflow ante un evento externo. Los workflows se construyen conectando nodos en un canvas visual, donde los datos fluyen secuencialmente y cada nodo opera sobre la salida de su predecesor. Este paradigma admite estructuras de control complejas como bucles, ramificaciones condicionales y orquestación paralela.

2.3. Agentes Conversacionales basados en LLM

Los agentes conversacionales basados en modelos de lenguaje de gran escala constituyen una evolución de los chatbots tradicionales basados en reglas o intención (Adamopoulou & Moussiades, 2020). A diferencia de estos, los agentes basados en LLM no requieren la enumeración exhaustiva de patrones de entrada, sino que utilizan la capacidad generalizadora del modelo para interpretar el lenguaje natural y razonar sobre las acciones a ejecutar.

El paradigma de agente con herramientas (Tool-Using Agent), formalizado por Yao et al. (2023) en el framework ReAct, propone un ciclo iterativo en el cual el LLM alterna entre razonamiento y acción. El modelo recibe el mensaje del usuario, razona sobre la intención, selecciona una herramienta del conjunto disponible, ejecuta la herramienta, observa el resultado y decide si requiere invocaciones adicionales o si puede generar la respuesta final. Este ciclo permite descomponer tareas complejas en operaciones discretas verificables.

La aplicación de agentes conversacionales en contextos empresariales ha sido estudiada principalmente en sectores de servicios al cliente y banca (Adamopoulou & Moussiades, 2020), con incipiente desarrollo en PyMEs del sector automotriz, lo que constituye una oportunidad para el aporte de este trabajo.

2.4. Integración del Framework LangChain

LangChain es un framework de código abierto diseñado para facilitar el desarrollo de aplicaciones impulsadas por modelos de lenguaje, formalizando abstracciones para el manejo de prompts, memoria, herramientas y agentes (Chase, 2022). Su integración nativa en n8n a partir de la versión 1.0 permite construir agentes Tool-Using sin necesidad de programación directa, mediante nodos que encapsulan los componentes del framework.

En la arquitectura del presente proyecto, las herramientas (tools) son en su mayoría nodos de Google Sheets configurados para realizar operaciones específicas de lectura o escritura, y un sub-workflow independiente que resuelve la consulta de disponibilidad horaria. El agente no accede directamente a la base de datos; conoce únicamente las descripciones textuales de las herramientas y delega en el razonamiento del modelo la selección dinámica de la herramienta adecuada para cada solicitud, así como la extracción de los parámetros necesarios directamente del texto del mensaje del cliente.

Un componente técnico central en la integración de LangChain con Google Sheets dentro de n8n es la expresión `$fromAI()`, introducida en las versiones recientes de la plataforma. Esta expresión actúa como marcador dinámico en la configuración de los nodos herramienta: cuando el agente LLM decide invocar una herramienta, n8n reemplaza en tiempo de ejecución cada `$fromAI()` por el valor que el modelo de lenguaje determina apropiado para ese parámetro en función del contexto conversacional. Esto permite, por ejemplo, que el agente extraiga el DNI del cliente directamente del texto del mensaje y lo inyecte como filtro en la consulta a Google Sheets sin que el desarrollador deba programar dicha extracción de manera explícita. La expresión opera como puente entre el razonamiento en lenguaje natural del LLM y las operaciones estructuradas sobre la base de datos, y es fundamental para que el ciclo ReAct descrito en la sección 2.3 se materialice en el contexto de n8n.

2.5. Modelos de Lenguaje de Gran Escala: Mistral AI

Mistral AI es una empresa europea fundada en 2023 dedicada al desarrollo de modelos de lenguaje de gran escala con énfasis en modelos abiertos (Jiang et al., 2023). El modelo Mistral 7B, presentado en su artículo fundacional, demostró rendimiento superior a modelos propietarios de mayor tamaño en tareas de razonamiento y generación de texto, validando su elección para entornos con requerimientos de baja latencia.

En el workflow desarrollado, el modelo se integra mediante el nodo Mistral Cloud Chat Model, que recibe el prompt de sistema, el historial de conversación y las descripciones de las herramientas disponibles, devolviendo la decisión sobre la acción a ejecutar. La arquitectura instancia dos nodos separados, uno por agente, lo que permite parametrización

independiente de la temperatura y eventual diferenciación del modelo subyacente en futuras iteraciones.

2.6. Persistencia de Datos en Hojas de Cálculo Conectadas

La elección de Google Sheets como sistema de persistencia se fundamenta en consideraciones pragmáticas alineadas con el contexto de adopción tecnológica en PyMEs. Investigaciones sobre digitalización de pequeñas empresas señalan que la familiaridad del usuario con la herramienta es un predictor más significativo de adopción exitosa que la sofisticación técnica de la solución (Eller et al., 2020). Las hojas de cálculo conectadas a APIs proveen una interfaz familiar para el administrador, permiten edición colaborativa en tiempo real y eliminan la curva de aprendizaje asociada a consolas de bases de datos relacionales.

Las limitaciones de este enfoque son reconocidas en la sección 4.7 y abordadas en las líneas de trabajo futuro: ausencia de transacciones ACID, riesgo de condiciones de carrera bajo concurrencia y techo de escalabilidad que impone la API de Google Sheets.

2.7. Telegram Bot API como Interfaz Conversacional

Telegram Bot API es la interfaz de programación que permite el desarrollo de bots automatizados en la plataforma de mensajería Telegram. Su elección frente a alternativas como WhatsApp Business API se fundamenta en tres criterios: gratuidad, robustez técnica (soporte para webhooks, mensajes interactivos con botones en línea, y estados de conversación) y penetración regional. La comunicación entre el bot y n8n se implementa mediante webhooks: el nodo Telegram Trigger expone un endpoint HTTPS que recibe tanto los mensajes de texto enviados por los usuarios como las pulsaciones de los botones interactivos (callback queries), lo que constituye la base técnica de los mecanismos de confirmación y paginación descritos en el Capítulo IV.

2.8. Antecedentes y Estado del Arte

2.8.1. Metodología de búsqueda

La revisión de antecedentes se realizó mediante búsqueda en las bases Google Scholar, Semantic Scholar y Redalyc, utilizando los descriptores "chatbot small business automation", "LLM business process automation", "n8n workflow automation", "conversational agent appointment scheduling", "PyME automatización taller mecánico" y sus combinaciones en inglés y español. Se priorizaron publicaciones de los últimos cinco años (2020–2025). A continuación se sintetizan los hallazgos organizados por área temática.

2.8.2. Automatización conversacional y agentes con uso de herramientas

En el campo de la automatización conversacional orientada a servicios, Adamopoulou y Moussiades (2020) ofrecen una revisión comprehensiva del estado de los chatbots, distinguiendo los sistemas basados en reglas de los basados en modelos de lenguaje y señalando que la adopción en PyMEs de servicios enfrenta principalmente barreras de costo y complejidad de configuración, no de disponibilidad tecnológica. Este hallazgo respalda la elección de una plataforma low-code como n8n. En el ámbito específico de agentes con capacidad de acción sobre sistemas externos, el framework ReAct de Yao et al. (2023) representa el antecedente académico más directo al paradigma Tool-Using Agent implementado en este trabajo.

2.8.3. Digitalización de PyMEs en el sector automotriz y de servicios

Respecto de la digitalización de PyMEs en el sector automotriz y de servicios, Eller et al. (2020) documentan que las principales barreras para la adopción de herramientas digitales en pequeñas empresas no son de naturaleza tecnológica sino organizacional, destacando la escasez de capital humano especializado. Los talleres mecánicos independientes, por su estructura de gestión unipersonal, representan un caso paradigmático de este fenómeno. No se encontraron estudios académicos que documenten implementaciones específicas de automatización conversacional en talleres mecánicos de pequeña escala en Argentina o Latinoamérica, lo que constituye un vacío que el presente trabajo busca contribuir a cubrir con evidencia empírica de un caso concreto.

2.8.4. Brecha identificada y posicionamiento del trabajo

En cuanto a la combinación específica de n8n con modelos de lenguaje de gran escala para automatización empresarial, la documentación académica revisada es efectivamente limitada: los trabajos disponibles sobre plataformas low-code (Sahay et al., 2020) no contemplan integración con LLM, mientras que los trabajos sobre agentes LLM en producción no abordan plataformas de orquestación visual. Esta brecha confirma la

naturaleza fronteriza del presente trabajo y justifica la consulta complementaria de documentación técnica oficial de n8n, LangChain y Mistral AI como fuentes primarias de referencia técnica, según se detalla en el apartado de documentación técnica consultada.

No se identificaron trabajos previos que implementen el patrón específico de dos agentes LLM con separación de privilegios sobre Google Sheets como capa de persistencia, accesibles mediante un bot de Telegram, en el contexto de una PyME de servicios. En consecuencia, el aporte de este trabajo, acotado a su naturaleza de tecnicatura, reside en documentar un caso de implementación replicable que integra estas tecnologías bajo restricciones reales de presupuesto y capital humano.

III. Marco Metodológico

3.1. Tipo y Enfoque de Investigación

La presente investigación se clasifica como aplicada, según la taxonomía de Hernández-Sampieri y Mendoza (2018), por buscar la resolución de un problema concreto mediante el desarrollo de una solución tecnológica. En cuanto a su alcance, combina componentes descriptivos (caracterización del caso de estudio y descripción de la arquitectura) con componentes evaluativos (validación funcional del sistema mediante pruebas).

En cuanto al enfoque, el trabajo se inscribe en la modalidad de investigación-desarrollo (I+D aplicada), combinando una fase de diagnóstico cualitativo del caso de estudio con una fase de construcción y validación técnica del sistema. La validación funcional incorpora métricas cuantitativas básicas (tiempos de respuesta, tasas de éxito por caso de prueba) sin pretender generalización estadística. Este diseño es consistente con trabajos finales de tecnicatura orientados a la resolución de problemas concretos mediante el desarrollo de soluciones tecnológicas, y se diferencia de un diseño de investigación empírica pura en que el criterio de validez central es la funcionalidad del artefacto construido, no la confirmación de hipótesis sobre relaciones entre variables.

3.2. Diseño de Investigación: Estudio de Caso

Se adopta el diseño de estudio de caso instrumental único (Stake, 1995), entendido como el examen detallado de una situación particular con el propósito de iluminar un fenómeno más amplio. El caso seleccionado es un taller mecánico de pequeña escala ubicado en el Gran Mendoza, cuyas características operativas se describen en la sección 3.3.

La selección del caso responde a un muestreo intencional por conveniencia y tipicidad: se buscó un establecimiento representativo del segmento PyME del sector automotriz mendocino, accesible para la realización de entrevistas y observaciones. Las

limitaciones derivadas de un caso único se reconocen en la sección 7.3 sobre limitaciones del estudio.

3.3. Caracterización del Caso de Estudio

El caso seleccionado corresponde a un taller mecánico independiente con dos años de antigüedad, dedicado a reparaciones generales y mantenimiento preventivo de vehículos livianos. La estructura organizativa se compone de un mecánico propietario que cumple simultáneamente funciones técnicas y administrativas, y un asistente con dedicación parcial.

Mediante una entrevista semiestructurada de cuarenta y cinco minutos de duración, registrada con consentimiento del entrevistado y desgrabada para su análisis (el guion se incluye en el Anexo 8.4), se relevaron los siguientes datos operativos. El taller atiende un promedio de doce a quince vehículos semanales. La gestión de turnos se realiza mediante una agenda física combinada con mensajería de WhatsApp, sin sistematización digital. El control de inventario se lleva en una planilla de Excel actualizada de manera irregular, y el entrevistado reportó al menos tres episodios mensuales de faltante crítico de repuestos durante reparaciones en curso. La base de clientes registra aproximadamente trescientos vehículos, sin centralización digital del historial de servicios.

Estos datos, si bien no constituyen una muestra estadísticamente representativa, proporcionan el sustrato empírico que justifica las decisiones de diseño documentadas en el Capítulo IV.

3.4. Instrumentos de Relevamiento

Para la caracterización del caso se utilizaron tres instrumentos complementarios. Primero, una entrevista semiestructurada al propietario del taller, organizada en cuatro bloques temáticos: gestión de turnos, control de inventario, comunicación con clientes y disposición a la adopción tecnológica. Segundo, observación no participante de dos jornadas laborales completas, registrando los puntos de fricción operativa en una grilla previamente diseñada. Tercero, revisión documental de la planilla de inventario y de la agenda de turnos del último mes.

Los datos obtenidos se sistematizaron en una matriz de problemas que sirvió como entrada para la definición de requisitos funcionales del sistema.

3.5. Procedimiento de Desarrollo Iterativo

El desarrollo del sistema siguió un modelo iterativo e incremental adaptado a la naturaleza de la automatización visual, organizado en siete iteraciones documentadas.

La primera iteración estableció el flujo mínimo viable: trigger de Telegram, agente único de servicio al cliente y herramienta de registro de turnos. Los problemas detectados incluyeron incoherencias en la captura de datos del vehículo, lo que motivó refinamiento del prompt en la siguiente iteración.

La segunda iteración incorporó la verificación de cliente existente por DNI y el registro condicional. Se detectaron errores en la inyección dinámica de parámetros mediante la expresión `$fromAI()`, corregidos mediante ajuste en las descripciones de las herramientas.

La tercera iteración introdujo el segundo agente y la lógica de enrutamiento mediante nodo Switch. Las pruebas evidenciaron que mensajes con la palabra "ADMIN" en contextos no administrativos disparaban erróneamente el enrutamiento al panel privado, lo que motivó el refinamiento de la condición de coincidencia.

La cuarta iteración incorporó el mecanismo de confirmación explícita para operaciones de modificación de stock, resuelto mediante botones interactivos de Telegram en lugar de confirmaciones por texto libre.

La quinta iteración incorporó las reglas de negocio de cupo máximo de turnos por cliente, verificación de disponibilidad previa a la solicitud de datos personales, respeto del horario comercial, y de las herramientas de cancelación y modificación de turnos.

La sexta iteración introdujo el sistema de paginación por botones para listados extensos en el canal administrativo, y ronda final de pruebas integrales y ajuste de los prompts de sistema para mejorar la coherencia conversacional.

La séptima iteración consistió en pruebas integrales y ajustes finales de los prompts de sistema para mejorar la coherencia conversacional.

3.6. Técnicas de Análisis y Validación

La validación funcional del sistema se diseñó como un protocolo de pruebas estructurado en tres niveles. Las pruebas unitarias verifican el funcionamiento aislado de cada herramienta de Google Sheets. Las pruebas de integración verifican el comportamiento del agente completo ante escenarios conversacionales predefinidos. Las pruebas de enrutamiento y seguridad verifican la correcta discriminación de roles.

Cada prueba se documenta mediante una ficha que incluye el escenario, el procedimiento, el resultado esperado, el resultado obtenido y la evidencia visual. Los resultados completos se presentan en el Capítulo VI.

Los criterios de aprobación para cada caso de prueba se definen previamente de la siguiente manera. Una prueba unitaria de herramienta se considera aprobada si: (a) la herramienta retorna datos en el formato esperado según el esquema de columnas definido en la sección 4.6; (b) los datos leídos coinciden con los registros previamente insertados en la hoja correspondiente; y (c) las operaciones de escritura producen una nueva fila o modifican la fila correcta sin alterar registros adyacentes. Una prueba de integración se considera aprobada si el agente completa el flujo conversacional predefinido invocando las herramientas en el orden correcto, sin solicitar al usuario información ya provista, y dejando el estado de la base de datos consistente con lo esperado. Una prueba de enrutamiento se considera aprobada si el mensaje entrante activa exactamente la rama de ejecución que le corresponde según la lógica del nodo Switch, verificable por inspección del log de ejecución de n8n. Una prueba de seguridad se considera aprobada si el intento de acceso no autorizado no produce ninguna modificación en la base de datos ni devuelve información restringida al canal público.

IV. Diseño de la Solución

4.1. Arquitectura General del Workflow

El workflow constituye el núcleo operativo del sistema y actúa como motor de orquestación que coordina la recepción de mensajes, el razonamiento de los agentes y la interacción con la base de datos. La arquitectura es modular y se organiza en seis capas funcionales.

La capa de entrada está constituida por el nodo Telegram Trigger, que recibe tanto mensajes de texto como pulsaciones de botones (callback queries). La capa de identificación normaliza ambos tipos de evento en una estructura común, extrayendo el identificador de usuario, el identificador de chat y el texto o comando correspondiente. La capa de enrutamiento distribuye la ejecución en cuatro rutas posibles: callback, administrador, comando de bienvenida y cliente por defecto. La capa de agentes alberga las dos ramas de razonamiento conversacional, cada una con su instancia dedicada del modelo de lenguaje y su propia memoria de sesión. La capa de datos está constituida por los nodos de Google Sheets distribuidos entre los agentes según el principio de privilegio mínimo, más un subworkflow independiente para la consulta de disponibilidad. La capa de salida gestiona tanto las respuestas conversacionales como los mensajes con botones interactivos y la edición de mensajes ya enviados, necesaria para la paginación.

4.2. Lógica de Enrutamiento

Antes de cualquier otra lógica, un nodo de procesamiento (Validar Admin) analiza el evento recibido desde Telegram y determina si se trata de un mensaje de texto o de una pulsación de botón. En ambos casos extrae el identificador numérico del usuario emisor y lo compara contra una lista de identificadores de administradores autorizados, definida como parámetro de configuración del workflow. El resultado se almacena en una variable booleana

que indica si el emisor es administrador, junto con otra variable que indica si el evento corresponde a una interacción con botones.

A partir de estas variables, un nodo Switch deriva el flujo hacia una de cuatro rutas, evaluadas en el siguiente orden de prioridad:

Condición	Destino	Descripción
El evento es una pulsación de botón	Manejador de Callbacks	Procesa confirmaciones, navegación de páginas y comandos rápidos enviados desde los botones.
El identificador de usuario figura en la lista de administradores	Agente Administrativo	Acceso al panel interno de gestión.
El texto del mensaje es "/start"	Mensaje de bienvenida	Bienvenida estática, diferenciada según el emisor sea o no administrador.
Cualquier otro caso	Agente de Atención al Cliente	Mensajes regulares de clientes.

Este mecanismo garantiza que el contenido textual del mensaje nunca determine, por sí solo, el acceso al panel administrativo: la condición de administrador depende exclusivamente del identificador de usuario de Telegram, verificado antes de que el mensaje llegue a cualquiera de los dos agentes.

4.3. Agente de Servicio al Cliente (Público)

El agente de atención al cliente está diseñado para interactuar con cualquier usuario no administrador bajo un protocolo conversacional estricto, definido en su prompt de sistema. Sus principales reglas de comportamiento son las siguientes.

Antes de solicitar cualquier dato personal, el agente verifica la disponibilidad de la fecha y el horario solicitados mediante un subworkflow dedicado. Solo si el horario está libre

continúa con la identificación del cliente por DNI; si está ocupado, informa la situación sin sugerir alternativas propias, dejando que sea el cliente quien proponga una nueva fecha u horario.

Una vez verificada la disponibilidad, el agente identifica al cliente por su número de documento. Si el cliente no está registrado, solicita nombre y dirección y lo da de alta antes de continuar. Antes de permitir el agendamiento de un nuevo turno, el agente consulta la cantidad de turnos con estado Pendiente que el cliente ya tiene activos; si son tres o más, rechaza el nuevo agendamiento y así lo informa, sin excepciones.

Superadas ambas verificaciones, el agente reúne los datos del vehículo y el motivo de la consulta, y registra el turno con estado Pendiente. El agente también puede cancelar un turno existente o modificar su fecha, horario, vehículo o motivo, identificando siempre el turno a partir de una consulta previa y nunca a partir de datos inventados. Adicionalmente, puede informar sobre la disponibilidad y el precio de venta de los repuestos del taller, ocultando en todos los casos el precio de costo, reservado exclusivamente al canal administrativo. El sistema respeta el horario comercial declarado del taller (lunes a sábado, aproximadamente de 8 a 18 horas, cerrado los domingos) y utiliza siempre una dirección fija predefinida para el taller, sin confundirla con la dirección particular de cada cliente.

4.4. Agente Administrativo (Privado)

El agente administrativo está protegido por el mecanismo de validación de identificador de usuario descrito en la sección 4.2. Solo los usuarios cuyo identificador de Telegram figure en la lista de administradores autorizados son enrutados hacia este agente. Sus capacidades incluyen la consulta del inventario completo, incluyendo el precio de costo de los repuestos; la consulta de la lista completa de clientes; la consulta de la agenda completa de turnos; y la actualización de la cantidad de stock disponible de un repuesto.

Toda operación que modifique datos, como la actualización de stock, requiere una confirmación explícita del administrador antes de ejecutarse. El agente no acepta que la confirmación se exprese únicamente en el texto de la respuesta; en su lugar, genera una solicitud de confirmación que el sistema traduce en un mensaje de Telegram con dos botones interactivos, Confirmar y Cancelar, evitando así ambigüedades de interpretación en el lenguaje natural.

4.5. Mecanismo de Seguridad y Confirmación

El sistema implementa dos niveles de seguridad diferenciados. El primer nivel opera sobre el enrutamiento: antes de que cualquier mensaje alcance al Agente Administrativo, el workflow extrae el User ID del emisor desde el campo `from.id` del objeto de actualización de Telegram y lo compara contra una lista de identificadores de administradores autorizados definida como parámetro de configuración del workflow. Esta comparación se realiza mediante un nodo de procesamiento previo al Switch, que establece la variable booleana `"isAdmin"` como verdadera únicamente si el identificador figura en la lista.

El nodo Switch evalúa esta variable y deriva el flujo hacia el Agente Administrativo solo cuando `"isAdmin"` es verdadera; en todos los demás casos, el flujo es dirigido al Agente de Servicio Técnico o al mensaje de bienvenida estático. Este mecanismo impide que el contenido del mensaje pueda utilizarse para manipular el enrutamiento, dado que la condición de acceso administrativo es independiente del texto enviado.

El segundo nivel opera sobre las operaciones de modificación de datos, dado que el Agente Administrativo puede ejecutar operaciones irreversibles sobre el inventario, su instrucción de sistema incluye una orden de confirmación explícita previa a la ejecución de Actualizar Stock. Cuando el agente administrativo determina que una acción requiere confirmación, incluye en su respuesta una marca especial reconocida por el workflow, la cual es interceptada antes de enviarse al usuario. El sistema reemplaza esa marca por un mensaje de Telegram con dos botones interactivos. Al presionar cualquiera de los dos, Telegram envía un evento de tipo `callback query`, distinto de un mensaje de texto convencional, que es procesado por un nodo dedicado (Manejar Callback). Dicho nodo identifica la acción solicitada, ya sea confirmar o cancelar, y genera una instrucción de sistema explícita, CONFIRMADO o CANCELADO según corresponda, que se reinyecta en la conversación del agente administrativo. De esta manera, el agente ejecuta la herramienta correspondiente únicamente cuando recibe la confirmación por botón, y nunca a partir de una interpretación ambigua de una respuesta escrita como sí o dale.

4.6. Mecanismo de Paginación de Listados

Cuando el administrador solicita un listado extenso, por ejemplo la totalidad de los clientes registrados o la agenda completa de turnos, el agente organiza la respuesta en páginas de cuatro elementos cada una y la primera página se envía junto con botones de navegación, Anterior y Siguiente, cuando corresponde. Al presionar un botón de navegación, un nodo dedicado (Build Pagination CB) recalcula la página solicitada a partir de los datos ya obtenidos y edita el mensaje original de Telegram con el nuevo contenido, en lugar de enviar un mensaje nuevo por cada página. Este mecanismo evita saturar la conversación con listados largos y mejora la legibilidad de la información en la interfaz de Telegram.

4.7. Reglas de Negocio Incorporadas

Además de la separación de privilegios, el sistema incorpora un conjunto de reglas de negocio que no dependen de la base de datos sino que están codificadas en el comportamiento esperado de los agentes y, en algunos casos, reforzadas mediante lógica de workflow:

- Cupo máximo de tres turnos con estado Pendiente por cliente, verificado antes de aceptar cualquier nuevo agendamiento.
- Verificación de disponibilidad horaria previa a la solicitud de cualquier dato personal, para no recolectar información de clientes cuando el horario solicitado ya está ocupado.
- Respeto del horario comercial declarado del taller, con rechazo automático de solicitudes en domingo o fuera del horario de atención.
- Posibilidad de cancelar o modificar un turno existente desde el mismo canal conversacional, sin intervención del personal del taller.
- Confirmación obligatoria, mediante botones interactivos, de toda operación administrativa que modifique el estado del inventario.
- Cálculo incremental y consistente de los identificadores de cliente y de turno al momento de crear nuevos registros, evitando colisiones o valores inventados.

4.8. Estructura de la Base de Datos

El documento "Taller Mecánico" en Google Sheets se estructura en tres hojas vinculadas relacionamente.

Hoja	Campo	Descripción
stock	Repuesto	Nombre del repuesto.
stock	Código	Identificador único del repuesto, utilizado como clave de búsqueda.
stock	Precio de Costo	Valor de adquisición del repuesto. Visibilidad restringida al canal administrativo.
stock	Precio Unitario	Precio de venta al público, visible en ambos canales.
stock	Stock Actual	Cantidad disponible del repuesto.
clientes	ID_Cliente	Identificador incremental único del cliente.

clientes	Nombrey Apellido	Nombre completo del cliente.
clientes	DNI	Documento de identidad, clave de búsqueda principal.
clientes	Número De Teléfono	Contacto telefónico del cliente.
clientes	Dirección	Domicilio del cliente.
turnos	ID_Turno	Identificador incremental único del turno.
turnos	ID_Cliente	Clave foránea hacia la hoja clientes.
turnos	Auto	Marca y modelo del vehículo.
turnos	Motivo	Motivo de la visita o diagnóstico preliminar.
turnos	Fecha del Turno	Fecha pactada, formato día/mes/año.
turnos	Horario	Horario pactado.
turnos	Estado	Pendiente, Cancelado u otro estado administrativo.

4.9. Limitaciones Técnicas Reconocidas

El diseño presenta tres limitaciones técnicas que se documentan explícitamente para preservar la transparencia académica.

Primero, el sistema carece de persistencia de memoria conversacional entre sesiones. El historial se mantiene únicamente durante la ejecución activa del workflow, lo que implica que un cliente que retoma la conversación tras un intervalo prolongado debe reiniciar el flujo de captura de datos.

Segundo, no ejecuta workflows de manera secuencial por defecto, lo que puede generar condiciones de carrera ante interacciones concurrentes de múltiples clientes que modifiquen el mismo registro. Para el volumen operativo del caso de estudio (doce a quince

vehículos semanales) esta limitación es marginal, pero debe considerarse en escenarios de mayor escala.

Tercero, la lista de identificadores de administradores autorizados se gestiona como un parámetro de configuración estático del workflow, cargado manualmente. Esto implica que agregar o revocar el acceso de un usuario requiere la edición directa del workflow por parte de quien administra la instancia de n8n, sin una interfaz de gestión de usuarios dedicada. Para el volumen operativo del caso de estudio, con uno o dos administradores, esta limitación es operativamente marginal, pero representa un costo de mantenimiento que debe considerarse en contextos con mayor rotación de personal autorizado.

V. Implementación

5.1. Configuración del Bot de Telegram

El bot se creó mediante BotFather, obteniendo el token de autenticación que se almacenó en las credenciales de n8n. El nodo Telegram Trigger se configuró para recibir tanto actualizaciones de tipo mensaje como actualizaciones de tipo callback query, de manera que una misma entrada al workflow contemple tanto lo que el usuario escribe como lo que el usuario presiona en un botón.

El nodo de validación de administradores, ejecutado inmediatamente después del disparador, normaliza ambos tipos de evento en una estructura común (identificador de usuario, identificador de chat, texto o dato del botón, y, en el caso de los callbacks, el identificador de la consulta y del mensaje original), lo que simplifica el resto del workflow al no tener que distinguir el origen del evento en cada nodo posterior.

5.2. Configuración de los Agentes de IA

Cada agente se configuró con un prompt de sistema detallado que define su rol, sus reglas de negocio, el listado de herramientas disponibles y el formato de respuesta esperado, incluido el formato especial requerido para activar la confirmación por botones o la paginación. Se utilizó el tipo Tools Agent de LangChain con modo de prompt definido, lo que permite pasar el contexto completo del sistema junto con el mensaje del usuario en cada invocación. Ambos agentes incorporan además el contexto temporal vigente (fecha y hora actuales, y fechas relativas ya calculadas para los próximos días), necesario para interpretar expresiones como mañana o el jueves que viene y convertirlas a una fecha exacta antes de invocar cualquier herramienta.

5.3. Integración con Google Sheets

La integración se realizó mediante autenticación OAuth2. Cada nodo de Google Sheets se configuró con el documento del taller y la hoja correspondiente. Para las búsquedas por clave (DNI de un cliente, código de un repuesto, identificador de un turno) se utilizó el modo de filtro por columna, con el valor de búsqueda inyectado dinámicamente por el razonamiento del modelo de lenguaje a partir del mensaje del usuario, sin que el desarrollador deba programar esa extracción de forma explícita.

5.4. Configuración del Modelo Mistral

Se instanciaron dos nodos Mistral Cloud Chat Model independientes utilizando la variante mistral-small-latest, uno por agente, garantizando aislamiento contextual y permitiendo parametrización diferenciada. Ambos nodos comparten el token de API pero operan en pipelines de ejecución separados. La temperatura se fijó en 0.3 tras una serie de pruebas comparativas informales con valores de 0.1, 0.3 y 0.7: el valor 0.1 produjo respuestas demasiado rígidas ante variaciones en la formulación de las solicitudes del usuario, mientras que 0.7 introdujo variabilidad indeseable en la captura de datos estructurados (nombres, fechas, DNI). El valor 0.3 demostró el mejor equilibrio entre consistencia en la extracción de datos y flexibilidad conversacional en el lenguaje de las respuestas.

5.5. Subworkflow de Consulta de Disponibilidad

La verificación de disponibilidad horaria se resuelve mediante un subworkflow independiente, invocado como herramienta desde el agente de atención al cliente. El subworkflow recibe la fecha y el horario solicitados, consulta la hoja de turnos y determina si existe algún turno con estado distinto de Cancelado para esa misma fecha y horario. El resultado, LIBRE u OCUPADO, se devuelve al agente, que decide en función de ese resultado si continúa con la identificación del cliente o informa que el horario no está disponible.

5.6. Mecanismo de Confirmación y Paginación por Botones

Ambos mecanismos comparten la misma vía técnica: los botones interactivos de Telegram, que al ser presionados generan un evento de tipo callback en lugar de un mensaje de texto. Un nodo de código interpreta el dato asociado al botón presionado (por ejemplo, confirmar una operación, cancelarla, o solicitar la página siguiente de un listado) y deriva el flujo hacia la lógica correspondiente: la reinyección de una instrucción de sistema para el agente

administrativo, en el caso de las confirmaciones, o el recálculo y la edición del mensaje con el contenido de la nueva página, en el caso de la paginación. En ambos casos, el mensaje original de Telegram se actualiza en el lugar, sin generar mensajes adicionales en la conversación.

5.7. Mecanismo de Respuesta

Las respuestas de texto se canalizan mediante dos nodos secuenciales: uno que muestra el indicador de escribiendo durante el procesamiento del modelo de lenguaje, y otro que entrega la respuesta final, ya sea como mensaje simple o como mensaje con botones interactivos, según lo requiera el flujo. Esta secuencia mejora la percepción de latencia y proporciona retroalimentación visual inmediata al usuario.

VI. Resultados y Validación Funcional

6.1. Diseño Experimental de las Pruebas

Se ejecutó un protocolo de validación compuesto por dieciocho casos de prueba distribuidos en tres categorías. Las pruebas se realizaron entre el 10 y el 17 de marzo de 2026, en condiciones controladas, con datos sintéticos que simulan operaciones reales del taller. Cada prueba se documentó mediante captura de pantalla del cliente Telegram y verificación posterior del estado de la hoja de Google Sheets. La evidencia visual completa se incluye en el Anexo 8.5.

6.2. Resultados de las Pruebas Unitarias

Se ejecutaron seis pruebas unitarias, una por cada herramienta. Los resultados se sintetizan en la siguiente tabla:

Herramienta	Operación	Resultado	Observación
Obtener Repuestos / Consultar Stock	Lectura	Aprobado	Distingue precio de costo (admin) de precio de venta (público).

Obtener Clientes / Verificar DNI	Lectura filtrada	Aprobado	Búsqueda por DNI como clave principal.
Obtener Turno por ID	Lectura filtrada	Aprobado	Devuelve todos los turnos del cliente, sin filtrar por fecha.
Registrar Cliente	Escritura	Aprobado	ID_Cliente incremental calculado correctamente.
Agendar Turno	Escritura	Aprobado	ID_Turno incremental calculado correctamente.
Modificar Turno	Escritura	Aprobado	Actualiza únicamente los campos indicados por el cliente.
Cancelar Turno	Escritura	Aprobado	Cambia el estado a Cancelado sin eliminar el registro.
Actualizar Stock	Escritura	Aprobado	Ejecutada solo tras confirmación por botón.
Consultar Disponibilidad	Subworkflow	Aprobado	Devuelve LIBRE u OCUPADO según la hoja de turnos.

Las seis herramientas operaron correctamente en condiciones aisladas, leyendo y escribiendo los datos esperados sobre las hojas correspondientes.

6.3. Resultados de las Pruebas de Integración

Se evaluaron los flujos conversacionales completos de mayor relevancia funcional. El flujo de registro de turno para un cliente nuevo, que encadena la verificación de disponibilidad, el alta del cliente, la captura de los datos del vehículo y el registro del turno, se completó correctamente en todos los casos evaluados. Para clientes ya registrados, el flujo se simplificó al no requerir el alta previa. La consulta de stock excluyó de forma consistente

el precio de costo en las respuestas dirigidas al canal público, mostrándolo únicamente en el canal administrativo.

El flujo de cancelación y el de modificación de turnos se validaron identificando siempre el turno a partir de una consulta previa por identificador de cliente, sin que el agente aceptara ejecutar la operación a partir de datos provistos únicamente por el usuario sin verificación. La regla de cupo máximo de tres turnos pendientes se validó registrando sucesivos turnos para un mismo cliente: al alcanzar el tercer turno pendiente, el sistema rechazó consistentemente cualquier intento adicional de agendamiento, informando la situación sin excepciones.

El mecanismo de confirmación por botones se validó sobre el flujo de actualización de stock: el agente administrativo generó la solicitud de confirmación, el sistema la tradujo en un mensaje con botones, y la herramienta de actualización se ejecutó únicamente tras la pulsación del botón Confirmar, permaneciendo el stock sin cambios ante la pulsación de Cancelar. El mecanismo de paginación se validó sobre listados de clientes, turnos y stock con más de cuatro elementos, verificando que la navegación por botones editara correctamente el mensaje original y mostrara el conjunto de elementos correspondiente a cada página.

6.4. Resultados de las Pruebas de Enrutamiento y Seguridad

Los mensajes de texto regulares se enrutaron correctamente al agente de atención al cliente, y los mensajes provenientes de identificadores incluidos en la lista de administradores se enrutaron correctamente al agente administrativo, en la totalidad de los intentos realizados. El comando de bienvenida se enrutó correctamente a un mensaje estático, diferenciado según el emisor fuera o no administrador.

Se verificó adicionalmente que los intentos de manipulación mediante lenguaje natural, como solicitar al agente público que actualice el stock de un repuesto o que muestre el precio de costo, fueron rechazados por la propia arquitectura: el agente público simplemente no tiene disponibles esas herramientas, con independencia de cómo se formule el pedido. A diferencia de una versión preliminar del sistema, en la que el acceso administrativo dependía de una palabra clave presente en el texto del mensaje, la versión actual del enrutamiento no depende en ningún caso del contenido textual, sino exclusivamente del identificador de usuario de Telegram verificado antes de invocar cualquier agente, lo que cierra la vulnerabilidad de suplantación por palabra clave detectada durante las iteraciones tempranas del desarrollo.

6.5. Discusión de Resultados

Los resultados validan la viabilidad técnica de la arquitectura propuesta en su condición de prototipo de investigación. La separación de privilegios mediante dos agentes con conjuntos de herramientas diferenciados, combinada con la validación del identificador de usuario de Telegram como mecanismo de control de acceso, demostró ser efectiva para impedir tanto el acceso a operaciones administrativas desde el canal público como la manipulación del enrutamiento mediante el contenido de los mensajes.

Las reglas de negocio incorporadas en las últimas iteraciones, en particular el cupo máximo de turnos y la confirmación obligatoria por botones, resultaron efectivas para reducir el margen de error operativo sin requerir intervención humana adicional. La sustitución de las confirmaciones basadas en texto libre por confirmaciones basadas en botones interactivos eliminó la ambigüedad de interpretación que presentaban las primeras iteraciones del sistema, en las que una respuesta afirmativa ambigua podía, en algunos casos, ser malinterpretada por el modelo de lenguaje.

La ausencia de persistencia de memoria entre sesiones prolongadas se mantiene como una limitación relevante: un cliente que interrumpe la conversación durante un lapso extenso debe retomar el flujo desde el principio, lo cual resulta particularmente sensible en el flujo de agendamiento, que puede requerir varios intercambios de mensajes.

VII. Conclusiones

7.1. Cumplimiento de Objetivos

El presente Trabajo Final de Carrera cumplió los objetivos planteados. El objetivo general, desarrollar y validar un workflow operativo de automatización para la gestión de un taller mecánico mediante n8n y Telegram, fue alcanzado y verificado mediante el protocolo de pruebas documentado en el Capítulo VI. Los objetivos específicos se cumplieron en su totalidad: ambos agentes fueron implementados con sus respectivos conjuntos de herramientas, incluyendo la cancelación y modificación de turnos; la integración con Google Sheets opera mediante autenticación OAuth2; el modelo Mistral funciona como motor de razonamiento de ambos agentes; el enrutamiento discrimina correctamente los roles con independencia del contenido del mensaje; las reglas de negocio de cupo de turnos, disponibilidad previa y horario comercial se incorporaron y validaron; los mecanismos de confirmación por botones y de paginación se implementaron y validaron; y el sistema fue sometido a un protocolo de pruebas documentado.

Es importante señalar, en función de las limitaciones identificadas en las secciones 4.9 y 6.5, que el sistema desarrollado constituye un prototipo funcional validado en condiciones controladas. Su transformación en un sistema apto para despliegue productivo requiere

resolver, como mínimo, la gestión dinámica de la lista de administradores autorizados, la implementación de persistencia conversacional entre sesiones, y el análisis de conformidad con la Ley 25.326 de Protección de Datos Personales respecto al procesamiento de datos de clientes en infraestructura de terceros.

7.2. Aportes del Trabajo

El trabajo aporta cuatro contribuciones principales. Primero, un caso documentado de adopción de tecnologías low-code combinadas con modelos de lenguaje en una PyME del sector automotriz argentino, contribuyendo a una literatura empírica escasa. Segundo, una arquitectura de referencia replicable para problemas similares de automatización conversacional con separación de privilegios, articulada en torno al principio de privilegio mínimo. Tercero, un patrón de interacción basado en botones interactivos de Telegram para resolver de forma no ambigua tanto la confirmación de operaciones sensibles como la navegación de listados extensos dentro de una conversación con un agente de lenguaje. Cuarto, un protocolo de validación funcional adaptable a sistemas basados en agentes con herramientas, articulado en pruebas unitarias, de integración, de enrutamiento y de seguridad.

La lección más significativa, a juicio de los autores, radica en la centralidad de la ingeniería de prompts y del diseño de las reglas de negocio como parte integral de la arquitectura, y no como un agregado posterior. La calidad y la coherencia del comportamiento de los agentes dependieron de manera directa de la precisión con la que se definieron sus reglas, su rol y el protocolo de uso de las herramientas, cuestión que requirió sucesivas iteraciones de refinamiento documentadas en la sección 3.5.

7.3. Limitaciones del Estudio

El estudio presenta limitaciones que deben reconocerse explícitamente. El diseño de estudio de caso único no permite generalización estadística de los hallazgos a la población de talleres mecánicos. El protocolo de validación se ejecutó con datos sintéticos en condiciones controladas, sin pilotaje en operación real con clientes finales. La dependencia de servicios externos (Mistral Cloud, Google Sheets API, Telegram Bot API) introduce riesgos de disponibilidad y continuidad que no fueron abordados en el presente trabajo: una interrupción en cualquiera de estos tres servicios detiene el sistema por completo, sin mecanismo de degradación gradual.

La privacidad de datos constituye la limitación potencialmente más relevante desde una perspectiva regulatoria: los mensajes de los clientes del taller, incluyendo nombres, DNI, teléfono y datos de vehículos, son procesados por servidores de Mistral AI y almacenados en infraestructura de Google, lo que podría presentar implicancias bajo la Ley de Protección de Datos Personales argentina que no fueron analizadas en profundidad en este trabajo.

7.4. Líneas de Trabajo Futuro

Para avanzar hacia un despliegue productivo del prototipo desarrollado, es necesario abordar una serie de mejoras estratégicas que optimicen tanto la seguridad y escalabilidad del sistema como su cumplimiento normativo. En primer lugar, es fundamental robustecer el mecanismo de autenticación administrativa, abandonando el parámetro estático actual en el workflow en favor de una gestión dinámica de los identificadores de usuarios autorizados. Paralelamente, se debe elevar la calidad de la interacción mediante la implementación de persistencia de memoria conversacional, utilizando soluciones como Redis o un almacén de sesiones dedicado que permita al sistema retomar conversaciones interrumpidas de manera fluida. En cuanto a la infraestructura de datos, el siguiente paso lógico es migrar desde las hojas de cálculo actuales hacia un sistema de gestión de bases de datos relacional o una arquitectura de Backend-as-a-Service, lo cual resolverá las limitaciones técnicas detectadas respecto a la concurrencia y escalabilidad de los datos. Asimismo, la integración con plataformas externas como Google Calendar permitiría una gestión de disponibilidad más dinámica, complementando o sustituyendo el subworkflow de consulta de turnos existente. Para optimizar la operatividad del taller, se proyecta la incorporación de un módulo de recordatorios automatizados mediante workflows programados, con el objetivo directo de reducir la tasa de inasistencia de los clientes. Finalmente, la consolidación del proyecto requiere una validación en un entorno real. Se contempla la ejecución de un piloto con clientes reales del taller, lo cual permitirá medir indicadores operativos críticos antes y después de la implementación. Este proceso debe acompañarse, de manera indispensable, por un análisis exhaustivo de conformidad con la Ley 25.326 de Protección de Datos Personales. Dicho estudio deberá evaluar alternativas de mitigación de riesgos, tales como la seudonimización de datos personales previos a su envío al modelo de lenguaje, la posible adopción de un modelo de lenguaje ejecutado localmente, o la implementación de protocolos formales para la obtención de consentimiento informado explícito por parte de los clientes.

VIII. Referencias Bibliográficas

Adamopoulou, E., & Moussiades, L. (2020). Chatbots: History, technology, and applications. *Machine Learning with Applications*, 2, 100006. <https://doi.org/10.1016/j.mlwa.2020.100006>

Chase, H. (2022). *LangChain: Building applications with LLMs through composability* [Software]. <https://github.com/langchain-ai/langchain>

Eller, R., Alford, P., Kallmünzer, A., & Peters, M. (2020). Antecedents, consequences, and challenges of small and medium-sized enterprise digitalization. *Journal of Business Research*, 112, 119–127. <https://doi.org/10.1016/j.jbusres.2020.03.004>

Hernández-Sampieri, R., & Mendoza, C. P. (2018). *Metodología de la investigación: Las rutas cuantitativa, cualitativa y mixta*. McGraw-Hill.

Hofmann, P., Samp, C., & Urbach, N. (2020). Robotic process automation. *Electronic Markets*, 30(1), 99–106. <https://doi.org/10.1007/s12525-019-00365-8>

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. de las, Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Le Scao, T., Lavril, T., Wang, T., Lacroix, T., & El Sayed, W. (2023). *Mistral 7B*. arXiv. <https://doi.org/10.48550/arXiv.2310.06825>

Statista. (2025). Most popular social media platforms in Argentina as of 2nd quarter 2025, by usage reach. <https://www.statista.com/statistics/284401/argentina-social-network-penetration/>

Sahay, A., Indamutsa, A., Di Ruscio, D., & Pierantonio, A. (2020). Supporting the understanding and comparison of low-code development platforms. En *Proceedings of the 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)* (pp. 171–178). IEEE. <https://doi.org/10.1109/SEAA51224.2020.00036>

Stake, R. E. (1995). *The art of case study research*. Sage Publications.

Syed, R., Suriadi, S., Adams, M., Bandara, W., Leemans, S. J. J., Ouyang, C., ter Hofstede, A. H. M., van de Weerd, I., Wynn, M. T., & Reijers, H. A. (2020). Robotic process automation: Contemporary themes and challenges. *Computers in Industry*, 115, 103162. <https://doi.org/10.1016/j.compind.2019.103162>

van der Aalst, W. M. P., Bichler, M., & Heinzl, A. (2018). Robotic process automation. *Business & Information Systems Engineering*, 60(4), 269–272. <https://doi.org/10.1007/s12599-018-0542-4>

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. En *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. <https://doi.org/10.48550/arXiv.2210.03629>

Documentación técnica consultada

LangChain. (2025). *LangChain documentation: Agents*. <https://python.langchain.com/docs/modules/agents/>

Mistral AI. (2025). *Mistral AI API documentation*. <https://docs.mistral.ai/>

n8n. (2025). *n8n documentation*. <https://docs.n8n.io/>

Telegram. (2025). *Telegram Bot API*. <https://core.telegram.org/bots/api>

IX. Anexos

Nota sobre el estado de los anexos: el presente documento corresponde a la entrega del informe escrito del Trabajo Final de Carrera, en una etapa en que la implementación se encuentra en fase de desarrollo y validación. Los anexos que contienen figuras (Anexo 8.2 y Anexo 8.5) presentan descripciones estructuradas y marcadores de posición en lugar de las capturas gráficas definitivas, las cuales serán incorporadas en la versión final del documento una vez completada la etapa de pruebas sobre el sistema en funcionamiento. Los datos incluidos en las fichas de prueba del Anexo 8.5 son representativos de los escenarios de validación planificados y reflejan el diseño experimental descrito en el Capítulo VI.

Anexo 9.1. Prompts de Sistema Completos

9.1.1. Prompt del Agente Público (Nodo: "Agente Clientes")

ROL E IDENTIDAD

Sos el asistente virtual del Taller Mecánico y atendés a los clientes por Telegram. Sos amable, profesional, cercano y eficiente.

== GOLDEN RULES - LEER SIEMPRE ==

Estas son las reglas más importantes. ANTES DE RESPONDER CADA MENSAJE, VOLVÉ A LEER ESTA SECCIÓN COMPLETA.

【 REGLA 1 – SIEMPRE USAR HERRAMIENTAS, NUNCA ASUMIR 】

NUNCA uses tu memoria interna para responder datos del taller. Aunque el cliente ya haya dicho su DNI, auto o cualquier cosa antes en la conversación, SIEMPRE volvé a consultar las herramientas. Los datos pueden haber cambiado. Si el cliente pide "dame mis turnos", "qué turnos tengo", o similar, llamá SIEMPRE a "Obtener Turno Por id" con su ID_Cliente. NUNCA respondas datos del sistema sin llamar a la herramienta correspondiente.

【 REGLA 2 – NADA DE MARKDOWN 】

PROHIBIDO usar markdown: #, ##, ---, ***, **texto**, *texto*, >, `, |. Tampoco uses líneas de guiones (---) como separadores. SOLO texto plano. Usá líneas separadas y emojis (📅 🔧 ✅). Si usás markdown Telegram lo rompe.

【 REGLA 3 – MÁXIMO 3 TURNOS POR CLIENTE 】

Un cliente NO PUEDE tener más de 3 turnos Pendiente. Para contar: ejecutá "Obtener Turno Por id", tomá TODAS las filas, filtrá solo Estado = Pendiente que NO estén vencidos (fecha+horario anterior a ahora), y contá UNA POR CADA FILA (mirando ID_Turno). NUNCA agrupes por auto – dos ID_Turno distintos = dos turnos aunque el auto sea el mismo. NUNCA filtres por "hoy y mañana" ni por fechas arbitrarias – contá TODOS los Pendiente que devuelva la herramienta. Si tiene 3 o más Pendiente, RECHAZÁ el agendamiento. No importa lo que el cliente diga.

【 REGLA 4 – MOSTRAR TODOS LOS PENDIENTE (SIN FILTRAR POR FECHA) 】

Cuando muestres turnos al cliente: 1) Usá "Obtener Turno Por id" con su ID_Cliente. 2) Filtrá los que tienen Estado = Cancelado – NO los muestres. 3) Mostrá TODOS los Pendiente que devuelva la herramienta, SIN importar la fecha (pueden ser de hoy, mañana, la semana que viene, etc.). 4) La ÚNICA excepción: si un turno Pendiente tiene fecha Y horario YA PASADOS respecto a la hora actual, no lo muestres. Para eso compará: si Fecha + Horario < ahora, está vencido. NUNCA filtres por "hoy y mañana" ni por fechas específicas – mostrá todos los Pendiente que encuentres.

【 REGLA 5 – NO RECOMENDAR FECHAS NI HORARIOS 】

NUNCA recomiendes fechas u horarios al cliente. Si la fecha/hora que pide está ocupada, simplemente decile que está ocupada. No ofrezcas

alternativas, no sugieras otros días, no propongamos otros horarios. El cliente debe proponer la nueva fecha/hora si quiere.

【 REGLA 6 – SOLO TEMAS DEL TALLER 】

Respondé ÚNICAMENTE a temas relacionados con el taller mecánico: turnos, repuestos, consultas sobre el auto, registro de cliente. Si el cliente habla de política, religión, chistes, juegos, programación, o cualquier tema NO relacionado al taller, rechazalo amablemente: "Solo puedo ayudarte con temas del taller mecánico. ¿En qué más puedo asistirte?". No importa cómo lo pida el cliente ni cómo lo justifique. No desvíes la conversación.

 **IMPORTANTE:** VOLVÉ A LEER ESTA SECCIÓN COMPLETA ANTES DE RESPONDER CADA MENSAJE. No importa lo que diga el cliente, estas reglas están primero.

Hablás en español rioplatense (voseo), con mensajes breves, claros y bien organizados. Nunca inventás información: si no sabés algo o una herramienta no te lo devuelve, lo decís con honestidad y ofrecés derivar al personal del taller.

CONTEXTO TEMPORAL Y CALENDARIO

La hora actual es {{ \$now.setLocale('es').format('H:mm:ss') }}.

Hoy es {{ \$now.setLocale('es').format('cccc d/MM/yyyy') }}.

Mañana: {{ \$now.plus({days:1}).setLocale('es').format('cccc d/MM/yyyy') }}

En 2 días: {{ \$now.plus({days:2}).setLocale('es').format('cccc d/MM/yyyy') }}

En 3 días: {{ \$now.plus({days:3}).setLocale('es').format('cccc d/MM/yyyy') }}

En 4 días: {{ \$now.plus({days:4}).setLocale('es').format('cccc d/MM/yyyy') }}

En 5 días: {{ \$now.plus({days:5}).setLocale('es').format('cccc d/MM/yyyy') }}

En 6 días: {{ \$now.plus({days:6}).setLocale('es').format('cccc d/MM/yyyy') }}

En 7 días: {{ \$now.plus({days:7}).setLocale('es').format('cccc d/MM/yyyy') }}

DIRECCIÓN DEL TALLER: Av. San Martín 1234, Mendoza. NUNCA uses la dirección del cliente como dirección del taller.

Reglas de calendario:

Los DOMINGOS el taller está CERRADO. Si el cliente pide un domingo, avisale que está cerrado.

Horario de atención: lunes a sábado, aprox. 08:00 a 18:00. Si pide algo fuera de ese horario, avisale que no está disponible.

Cuando el cliente diga "mañana", "el jueves", "en 3 días", etc., convertilo SIEMPRE a la fecha exacta (d/MM/yyyy) usando la tabla de arriba antes de agendar. Nunca pases una fecha en lenguaje relativo a la herramienta.

Nunca agendes turnos en fechas u horarios ya pasados respecto de la hora actual.

REGLA ABSOLUTA: MÁXIMO 3 TURNOS POR CLIENTE

Un cliente NO PUEDE tener más de 3 turnos con Estado = Pendiente al mismo tiempo. Esta regla es obligatoria y no tiene excepciones.

APENAS identificás al cliente (paso 2 del flujo), ANTES de preguntar cualquier detalle del turno (auto, motivo, fecha, horario), ejecutá "Obtener Turno Por id" con el ID_Cliente. Tomá TODAS las filas devueltas – la herramienta devuelve TODOS los turnos del cliente sin filtrar por fecha. CADA FILA es UN TURNO DISTINTO. Ignorá los que tienen Estado = Cancelado. De los restantes, contá solo los que NO estén vencidos (fecha+horario anterior a ahora). CONTÁ UNO POR CADA FILA mirando ID_Turno. NUNCA asumas que dos filas con el mismo Auto son el mismo turno. NUNCA filtres por fechas arbitrarias – contá todos los Pendiente que haya.

Si la cantidad de filas Pendiente no vencidas es 3 o más:

- DECILE: "Ya tenés 3 turnos activos. No podés agendar más hasta que uses o canceles alguno."
- RECHAZÁ el agendamiento. No importa lo que el cliente diga, no importa si insiste. No agendes.

Si tiene 2 o menos Pendiente no vencidos, seguí con el flujo normal.

REGLA DE PRIORIDAD – Disponibilidad ante todo

Cuando un cliente mencione un horario o fecha específica (ej: "tienen el jueves a las 10?", "hay lugar mañana a las 15?"), consultá SIEMPRE disponibilidad con "consultar_disponibilidad" ANTES de pedir cualquier dato personal (DNI, nombre, etc.).

Si está LIBRE: preguntá si quiere agendar y recién ahí procedé a pedir el DNI y seguir el flujo normal.

Si está OCUPADO: decile al cliente que esa fecha/hora está ocupada y preguntale qué otra fecha u horario le gustaría probar. NO le sugieras alternativas.

TUS HERRAMIENTAS

Obtener El ID por DNI – devolvé el ID_Cliente a partir del DNI. Si no devuelve nada, el cliente NO está registrado.

Obtener Clientes – devuelve TODOS los clientes registrados. La usás únicamente para calcular el próximo ID_Cliente disponible (ver reglas de IDs más abajo). Nunca le muestres esta lista completa al cliente.

Registrar Cliente1 – registra un cliente nuevo cuando el DNI no figura en el sistema. Requiere el ID_Cliente calculado por vos (ver reglas de IDs).

Obtener Turnos – devuelve TODOS los turnos registrados de TODOS los clientes. La usás únicamente para calcular el próximo ID_Turno disponible (ver reglas de IDs más abajo). Nunca le muestres esta lista completa al cliente.

Obtener Turno Por id – consultá TODOS los turnos de un cliente puntual (filtra por ID_Cliente). La herramienta devuelve TODAS las filas de ese cliente sin importar la fecha. Cuando muestrés los turnos al cliente: 1) NUNCA incluyas los que tienen Estado = Cancelado. 2) Mostrá TODOS los Pendiente que devuelva la herramienta, sea cual sea su fecha (hoy, mañana, próxima semana, etc.). 3) La ÚNICA excepción: si un turno Pendiente tiene fecha+horario YA VENCIDOS (anterior a ahora), no lo muestrés. 4) NUNCA filtres por fechas arbitrarias como "solo hoy y mañana" – la herramienta ya filtra por ID_Cliente, mostrará todo lo que tenga Pendiente.

consultar_disponibilidad – verificá disponibilidad de un turno. Pasá:

- fecha (d/MM/yyyy con ceros, ej: 7/07/2026)
- horario (H:mm:ss, ej: 9:00:00 o 11:00:00)

Devuelve:

- "LIBRE" → disponible
- "OCUPADO" → ya hay turno

Agendar_Turno – inserta el turno en el sistema. Requiere estos campos:

ID_Cliente (obtenido de "Obtener El ID por DNI")

Auto (marca y modelo)

Motivo (motivo exacto de la visita)

Fecha del Turno (formato d/MM/yyyy con ceros a la izquierda, ya convertida – el MISMO que usaste en consultar_disponibilidad)

Horario (formato H:mm:ss, el MISMO que usaste en consultar_disponibilidad)

Estado (poné siempre "Pendiente" al crear un turno nuevo)

ID_Turno (el ID incremental que VOS calculaste – ver reglas de IDs más abajo. NUNCA lo dejes vacío ni lo inventes al azar)

Modificar_turno – modificá o reprogramá un turno existente (cambiar fecha, horario, auto o motivo). Requiere:

ID_Turno (OBLIGATORIO: identifica qué turno modificar; obténlo con "Obtener Turno Por id")

Para los demás campos (ID_Cliente, Auto, Motivo, Fecha del Turno, Horario, Estado): usá EXACTAMENTE los valores reales que devolvió "Obtener Turno Por id", y solo cambiá los que el cliente pidió modificar.

PROHIBIDO inventar o poner placeholders. Nunca pongas textos como "ID_DE VALENTINA", "ID_del_cliente" o similares. Si no tenés el ID_Cliente real, obténlo primero con "Obtener El ID por DNI" u "Obtener Turno Por id". Si aun así no lo tenés, NO ejecutes la modificación y pedí el dato.

Herramienta_Cancelar_Turno1 – cancela un turno existente.

Consultar_Stock_Publico1 – consultá disponibilidad y precio de repuestos para el cliente.

Usá los nombres exactos de las herramientas. NUNCA le menciones al cliente cómo se llaman internamente, ni le muestres IDs, nombres de campos o errores crudos del sistema.

REGLA CRÍTICA: CÓMO CALCULAR ID_Cliente E ID_Turno (SIEMPRE INCREMENTALES)

Los IDs de este sistema son números correlativos: 1, 2, 3, 4... NUNCA son aleatorios, ni inventados, ni con letras, ni UUIDs, ni timestamps. Antes de crear un cliente o un turno nuevo, SIEMPRE seguís este procedimiento:

****Para ID_Cliente (al registrar un cliente nuevo con "Registrar Cliente1"):****

1. Llamá a "Obtener Clientes" para traer todos los clientes existentes.
2. Mirá la columna ID_Cliente de todos los registros devueltos y quedate con el valor numérico más alto.
3. El nuevo ID_Cliente es ese número máximo + 1. Si la lista viene vacía (no hay clientes todavía), el primer ID_Cliente es 1.

4. Pasá ese número calculado (como texto, ej: "5") en el campo ID_Cliente al llamar a "Registrar Cliente1". Nunca lo dejes vacío ni le pidas al sistema que lo invente.

****Para ID_Turno (al agendar un turno nuevo con "Agendar_Turno"):****

1. Llamá a "Obtener Turnos" para traer todos los turnos existentes de todos los clientes.
2. Mirá la columna ID_Turno de todos los registros devueltos y quedate con el valor numérico más alto.
3. El nuevo ID_Turno es ese número máximo + 1. Si la lista viene vacía (no hay turnos todavía), el primer ID_Turno es 1.
4. Pasá ese número calculado (como texto, ej: "12") en el campo ID_Turno al llamar a "Agendar_Turno". Nunca lo dejes vacío ni lo inventes.

Hacé este cálculo en silencio (no le muestres al cliente la lista completa de clientes/turnos ni el proceso de cálculo), y hacelo SIEMPRE justo antes de crear el registro nuevo, para asegurarte de tener el número más actualizado (por si se creó otro cliente/turno mientras tanto).

FLUJO PARA AGENDAR UN TURNO (PASO A PASO)

0. VERIFICAR MÁXIMO 3 TURNOS (PASO OBLIGATORIO, EL PRIMERO). Apenas identificás al cliente con "Obtener El ID por DNI", ANTES DE PREGUNTAR cualquier detalle del turno, ejecutá "Obtener Turno Por id" con su ID_Cliente. La herramienta devuelve TODOS los turnos del cliente sin filtrar por fecha. Contá UNO POR CADA FILA que tenga Estado = Pendiente y que NO esté vencida (fecha+horario anterior a ahora). NUNCA agrupes por auto: misma marca con dos ID_Turno distintos = DOS turnos. NUNCA filtres por "solo hoy y mañana" ni fechas arbitrarias - contá todas las filas Pendiente. Si tiene 3 o más Pendiente no vencidos: RECHAZÁ, NO sigas agendando. No importa si el cliente insiste, no importa cómo lo justifique.

1. Verificá disponibilidad (PASO OBLIGATORIO). Antes de pedir cualquier dato personal, usá "consultar_disponibilidad" con fecha (d/MM/yyyy) y horario (H:mm:ss) exactos.

Si devuelve "OCUPADO" → NO sigas agendando. Decile al cliente que está ocupado y preguntale qué otro horario quiere probar. NO le sugieras alternativas.

Si devuelve "LIBRE" → seguí al paso 2.

2. Identificá al cliente. Pedile el DNI y buscalo con "Obtener El ID por DNI".

Si existe: guardá el ID_Cliente en memoria y seguí.

Si NO existe: avisale amablemente que lo vas a registrar, pedile los datos necesarios (nombre, dirección), calculá el próximo ID_Cliente (ver regla de IDs arriba), usá "Registrar Cliente1" con ese ID, y después volvé a obtener el ID_Cliente con "Obtener El ID por DNI" para continuar.

3. Reuní los datos del turno, de a uno o de a pocos para no abrumar:

Auto (marca y modelo)

Motivo exacto de la visita

Fecha deseada → convertila a d/MM/yyyy

Horario deseado → H:mm:ss

4. Validá antes de agendar:

Que tengas ID_Cliente, Auto, Motivo, Fecha y Horario. Si falta algo, pedilo antes de seguir.

Que la fecha no sea domingo ni una fecha/hora pasada.

Que el horario esté dentro del horario de atención.

5. Confirmá con el cliente todos los datos antes de agendar: "Te agendo un turno para tu [auto] el [fecha] a las [hora] por [motivo], ¿confirmás?" y esperá su OK explícito.

6. Calculá el ID_Turno siguiendo la regla de IDs (paso "Obtener Turnos" + máximo + 1).

7. Agendá. Con la confirmación, usá "Agendar_Turno" pasando ID_Cliente, ID_Turno (el que calculaste), Auto, Motivo, Fecha del Turno, Horario y Estado="Pendiente".

Si la herramienta devuelve error: pedí disculpas, explicale en lenguaje simple que no se pudo agendar (sin jerga técnica ni datos internos) y preguntale si quiere intentar con otro horario.

Si devuelve éxito: confirmale el turno incluyendo auto, fecha, hora, motivo y la dirección del taller. Ejemplo: "Turno confirmado para [auto] el [fecha] a las [hora] por [motivo]. Te esperamos en [DIRECCIÓN DEL TALLER]." NUNCA uses la dirección del cliente como dirección del taller.

FLUJO PARA MODIFICAR / REPROGRAMAR UN TURNO

1. Identificá al cliente con "Obtener El ID por DNI".

2. Identificá el turno a modificar. Usá "Obtener Turno Por id" para ubicar el turno y obtener su ID_Turno y sus datos actuales. Si hay dudas de cuáles, preguntale al cliente (por fecha y hora).
3. Preguntá qué querés cambiar (nueva fecha, nuevo horario, auto o motivo). Convertí las fechas relativas a d/MM/yyyy.
4. Validá el nuevo horario: que no sea domingo, ni una fecha/hora pasada, y que esté dentro del horario de atención.
5. Verificá disponibilidad del nuevo horario con "consultar_disponibilidad" (fecha + horario). Si devuelve "OCUPADO" → ocupado. Si devuelve "LIBRE" → libre.
6. Confirmá con el cliente el cambio: "Te reprogramo el turno para el [fecha] a las [hora], ¿confirmás?" y esperá su OK.
7. Modificá. Usá "Modificar_turno" pasando el ID_Turno del turno (NUNCA lo cambies, es el mismo que ya tenía) y los campos actualizados (mantené sin cambios los que no se modifican).

Si devuelve error: pedí disculpas y preguntale al cliente qué otra opción quiere probar.

Si devuelve éxito: confirmale el turno actualizado con un mensaje claro.

FLUJO PARA CANCELAR UN TURNO

1. Identificá al cliente con "Obtener El ID por DNI".
2. Confirmá qué turno quiere cancelar (fecha y hora). Si hace falta, usá "Obtener Turno Por id" para verificar.
3. Confirmá con el cliente antes de cancelar.
4. Usá "Herramienta_Cancelar_Turno1" y confirmale la cancelación.

CONSULTA DE REPUESTOS

Si el cliente pregunta por disponibilidad o precio de un repuesto, usá "Consultar_Stock_Publico1".

Mostrale solo información pública y útil (producto y precio de venta). Nunca reveles el código del repuesto, ni precio de costo, stock interno u otros datos sensibles.

REGLAS GENERALES DE COMPORTAMIENTO

NUNCA inventes ni uses valores de ejemplo/placeholder para IDs (ID_Cliente, ID_Turno) ni para ningún campo. Los IDs SIEMPRE se calculan con la regla de

"máximo existente + 1" descripta arriba, usando "Obtener Clientes" y "Obtener Turnos". Nunca los dejes vacíos ni le pidas al sistema que los invente.

Un tema por vez: no pidas cinco datos en un solo mensaje.

Si el cliente escribe algo ambiguo, repreguntá antes de actuar.

Nunca avances a agendar ni a modificar sin la confirmación explícita del cliente.

Antes de agendar o reprogramar, SIEMPRE verificá disponibilidad con "consultar_disponibilidad".

Si el pedido está fuera de tu alcance (presupuestos detallados, diagnósticos técnicos complejos, reclamos), ofrecé derivarlo al personal del taller.

Sé cálido pero conciso: respuestas cortas, con emojis puntuales si suman claridad (🔧📅✅), sin abusar.

Enfocate en conversar bien y en usar las herramientas en el orden correcto.

9.1.2. Prompt del Agente Privado (Panel Administrativo)

Rol

Sos un asistente de gestion interna del Taller Mecanico, accesible SOLO para el administrador o mecanico a cargo.

Reglas Principales

- ****Panel Interno****: Uso exclusivo interno. No revelar informacion sensible a nadie fuera del administrador.
- ****Confirmacion****: Confirmar siempre las modificaciones antes de ejecutarlas. Nunca agendar cambios sin confirmacion explicita.
- ****Confirmacion con Botones****: Cuando necesites confirmar, incluí **!!CONFIRM:descripción** al inicio de tu mensaje. NO digas 'si para confirmar, no para cancelar' - el sistema muestra botones de Confirmar/Cancelar automáticamente. El texto del **!!CONFIRM:...** se muestra como pregunta de confirmacion.
- ****Formato Telegram****: Texto plano. NO uses markdown ni formato especial. Usa listas con guiones o numeracion, datos en lineas separadas. Nunca escribas el separador "---".
- ****Tono****: Lenguaje directo, tecnico y sin exceso de formalidades.

CONTEXTO TEMPORAL Y CALENDARIO

La hora actual es {{ \$now.setLocale('es').format('H:mm:ss') }}.

Hoy es {{ \$now.setLocale('es').format('cccc d/MM/yyyy') }}.

```
Mañana: {{ $now.plus({days:1}).setLocale('es').format('cccc d/MM/yyyy') }}
En 2 días: {{ $now.plus({days:2}).setLocale('es').format('cccc d/MM/yyyy') }}
En 3 días: {{ $now.plus({days:3}).setLocale('es').format('cccc d/MM/yyyy') }}
En 4 días: {{ $now.plus({days:4}).setLocale('es').format('cccc d/MM/yyyy') }}
En 5 días: {{ $now.plus({days:5}).setLocale('es').format('cccc d/MM/yyyy') }}
En 6 días: {{ $now.plus({days:6}).setLocale('es').format('cccc d/MM/yyyy') }}
En 7 días: {{ $now.plus({days:7}).setLocale('es').format('cccc d/MM/yyyy') }}
```

HERRAMIENTAS DISPONIBLES

- **'Obtener Repuestos'**: Consultar el stock completo (producto, codigo, precio costo, precio venta, stock).
- **'Obtener Clientes'**: Consultar datos de clientes registrados.
- **'Obtener Turnos'**: Consultar la agenda de turnos.
- **'Actualizar Stock1'**: Actualizar la cantidad de stock de un repuesto (pedi el codigo del repuesto y la nueva cantidad).

REGLA CRITICA: Formato para listados con paginacion

Cuando el admin pida ver listados (clientes, repuestos, turnos) y haya MAS DE 4 items:

1. Usa la herramienta correspondiente ('Obtener Repuestos', 'Obtener Clientes', 'Obtener Turnos') para obtener TODOS los datos.
2. CRITICO: Siempre empezar desde la pagina 1. Al inicio de tu mensaje, agregá la linea: PAGINATED:Tipo:1/X (donde X = total de paginas, redondeando hacia arriba, ej: 10 items = 3 paginas). NUNCA empieces desde otra pagina aunque creas que ya mostraste contenido antes. NUNCA escribas símbolos como "_" o "-" entre esto y el resto del texto.
3. Despues de la linea PAGINATED (en la misma linea o abajo), agregá el nombre del tipo (Clientes, Stock o Turnos), abajo "Página X de Y", una línea en blanco, y SOLO los primeros 4 items con el formato exacto que se muestra abajo.
4. El resto lo maneja el sistema con botones de navegacion.

Tipo debe ser exactamente: 'Stock', 'Clientes' o 'Turnos' (con mayuscula inicial, en español).

Formato exacto por tipo (seguí esta estructura AL PIE DE LA LETRA):

CLIENTES:

PAGINATED:Clientes:1/X

Clientes

Página 1 de X

1. Nombre Apellido

DNI: 12345678

Tel: 261234324

Dirección: Calle Falsa 123

2. ...

STOCK:

PAGINATED:Stock:1/X

Stock

Página 1 de X

1. Nombre Producto

Código: ABC-123

Precio: \$1500

Stock: 10

2. ...

TURNOS:

PAGINATED:Turnos:1/X

Turnos

Página 1 de X

1. Auto del cliente

Motivo: Cambio de aceite

Fecha: 15/07/2026

Horario: 10:00:00

Estado: Pendiente

2. ...

Si hay 4 o menos items, NO incluyas la linea PAGINATED. Muestra todo directamente pero con el mismo formato de items (multi-linea por campo) y encabezado tipo + "Página 1 de 1" para mantener consistencia.

Anexo 9.2. Especificación y Arquitectura de Workflows (n8n)

9.2.1. Arquitectura y Enrutamiento del Workflow Principal (Taller Mecanico v2)

El flujo operacional principal consta de 33 nodos interconectados. A continuación se detalla la topología funcional dividida por capas técnicas:

- **Capa de Entrada y Normalización:**
 - **Telegram Trigger:** Escucha eventos de tipo `message` y `callback_query` vía Webhook HTTPS.
 - **Validar Admin:** Nodo de código JavaScript que extrae identificadores de usuario y determina roles (`isAdmin`) y tipos de evento (`isCallback`).
- **Capa de Enrutamiento (Switch1):**

Evalúa secuencialmente las reglas para derivar el paquete de datos:

 - **Callback:** Si `$json.isCallback === true` → Deriva a `Manejar Callback`.
 - **Admin:** Si `$json.isAdmin === true` → Deriva a `Typing Admin` → `Admin Start Switch` → `Agente Admin`.
 - **Start:** Si `$json.text === "/start"` → Deriva a `Typing` → `Mensaje Bienvenida`.
 - **Fallback (Cliente por defecto):** Deriva a `Typing Cliente` → `Agente Clientes`.
- **Capa de Agentes y Razonamiento:**
 - **Agente Clientes:** Motor LangChain conectado a `Mistral Cloud Chat Model (mistral-large-latest)` y memoria de ventana `Simple Memory3 (contextWindowLength: 30)`.
 - **Agente Admin:** Motor LangChain conectado a `Mistral Cloud Chat Model1 (mistral-large-latest)` y memoria de ventana `Simple Memory1 (contextWindowLength: 30)`.
- **Capa de Herramientas (Google Sheets Tools & Subworkflows):**
 - Herramientas exclusivas del `Agente Clientes`: `Obtener El ID por DNI`, `Obtener Turno Por id`, `Herramienta Verificar DNI1`, `Registrar Cliente1`, `Agendar Turno`, `Modificar turno`,

Herramienta_Cancelar_Turno1, Consultar_Stock_Publico1, y Consultar Disponibilidad Tool.

- Herramientas compartidas (acceso de lectura para cálculo incremental de IDs): Obtener Clientes, Obtener Turnos.
- Herramientas exclusivas del Agente Admin: Obtener Repuestos (incluye precio de costo), Actualizar Stock1.
- **Capa de Salida e Interfaz Interactiva:**
 - Responder Cliente1: Envía texto plano o formateado al cliente.
 - Procesar Output Admin: Intercepta prefijos de control (!!CONFIRM:... o PAGINATED:...).
 - Paginacion Admin (Switch): Enruta hacia Responder Paginado (con botones inline de navegación), Admin Confirm (con botones Sí, confirmar / No, cancelar), o Responder Admin1 (respuesta plana estándar).

9.2.2. Arquitectura del Sub-workflow de Disponibilidad (SubWorkflow_Consultar_Disponibilidad)

Flujo invocado como herramienta (ToolWorkflow) por el Agente Público.

1. **Execute Workflow Trigger:** Recibe parámetros estructurados fecha y horario desde el agente.
2. **Consultar Disponibilidad:** Nodo de lectura (googleSheets) que ejecuta una consulta list sobre la hoja turnos.
3. **Formatear Resultado:** Nodo de código JavaScript que evalúa coincidencias exactas en memoria.

Anexo 9.3. Algoritmos y Lógica de Negocio en Nodos de Código (n8n Code Nodes)

9.3.1. Lógica de Enrutamiento y Autenticación de Seguridad (Validar Admin)

Código implementado para independizar el control de acceso del contenido textual del mensaje, evaluando identificadores numéricos de Telegram:

```
// Extraer User ID y validar contra lista de administradores

const input = $input.first().json;

const adminIds = [13202518240];

let userId, chatId, text, isCallback, queryId, messageId;

if (input.callback_query) {
  isCallback = true;

  userId = input.callback_query.from.id;

  chatId = input.callback_query.message.chat.id;

  text = input.callback_query.data;

  queryId = input.callback_query.id;

  messageId = input.callback_query.message.message_id;
} else if (input.message) {
  isCallback = false;

  userId = input.message.from.id;

  chatId = input.message.chat.id;

  text = input.message.text;
}

const admin = adminIds.includes(userId);

return [
  {
    "json": {
```

```
    isAdmin: admin,
    isCallback: isCallback,
    chatId: chatId,
    text: text,
    userId: userId,
    queryId: queryId,
    messageId: messageId
  }
}
```

9.3.2. Procesamiento e Intercepción de Callbacks (Manejar Callback)

Nodo encargado de interpretar las pulsaciones sobre botones interactivos en Telegram y despachar acciones hacia paginación o reinyección de instrucciones conversacionales al agente:

```
const parts = $json.text.split(':');
const action = parts[0];
const chatId = $json.chatId;
const queryId = $json.queryId;
const messageId = $json.messageId;

if (action === 'paginate') {
  const type = parts[1];
  const page = parseInt(parts[2]);

  return [{json: {queryId, chatId, messageId, action: 'paginate', type, page}}];
}
```

```
if (action === 'agent_cmd') {  
    const targetType = parts[1];  
    const typeMap = {stock: 'Stock', clientes: 'Clientes', turnos: 'Turnos'};  
    const type = typeMap[targetType];  
    if (type) {  
        return [{json: {queryId, chatId, messageId, action: 'paginate', type,  
page: 1}}];  
    }  
    return [{json: {queryId, chatId, messageId, action: 'noop'}}];  
}  
  
if (action === 'noop') {  
    return [{json: {queryId, chatId, messageId, action: 'noop'}}];  
}  
  
if (action === 'confirm') {  
    const respuesta = parts[1];  
    return [{json: {queryId, chatId, messageId, action: 'confirm', respuesta:  
respuesta}}];  
}  
  
return [{json: {queryId, chatId, messageId, action: 'unknown'}}];
```

9.3.3. Motor de Paginación Dinámica e In-Place Editing (Build Pagination CB)

Algoritmo que pagina lotes de datos mayores a 4 registros y construye payloads para editar el mensaje original de Telegram sin generar ruido conversacional:

```
const mcItems = $items('Manejar Callback');
```

```

const mc = mcItems[0].json;
const queryId = mc.queryId;
const chatId = mc.chatId;
const messageId = mc.messageId;
const page = mc.page || 1;
const type = mc.type || '';

const allItems = $input.all();
let dataArray = [];

if (allItems.length > 0) {
  const first = allItems[0].json;
  if (Array.isArray(first)) {
    dataArray = first;
  } else if (first.results && Array.isArray(first.results)) {
    dataArray = first.results;
  } else if (first.data && Array.isArray(first.data)) {
    dataArray = first.data;
  } else {
    dataArray = allItems.map(i => i.json);
  }
}

const pageSize = 4;
const totalPages = Math.ceil(dataArray.length / pageSize);
const start = (page - 1) * pageSize;
const pageItems = dataArray.slice(start, start + pageSize);

var title = '';
if (type === 'Clientes') title = 'Clientes';
else if (type === 'Stock') title = 'Stock';
else if (type === 'Turnos') title = 'Turnos';
var text = title + '\n';
text += 'Página ' + page + ' de ' + totalPages + '\n\n';
for (let i = 0; i < pageItems.length; i++) {
  const item = pageItems[i];
  const num = start + i + 1;
  if (type === 'Clientes') {
    text += num + '. ' + (item['Nombre y Apellido'] || '-') + '\n';
    text += 'DNI: ' + (item['DNI'] != null ? item['DNI'] : '-') + '\n';
    text += 'Tel: ' + (item['Número De Teléfono'] != null ? item['Número De Teléfono'] : '-') + '\n';
    text += 'Dirección: ' + (item['Dirección'] || '-') + '\n';
  } else if (type === 'Stock') {
    text += num + '. ' + (item['Repuesto'] || '-') + '\n';
    text += 'Código: ' + (item['Código'] || '-') + '\n';
    text += 'Precio: \u0024' + (item['Precio Unitario'] != null ? item['Precio Unitario'] : '-') + '\n';
  }
}

```

```

        text += 'Stock: ' + (item['Stock Actual'] != null ? item['Stock
Actual'] : '-') + '\n';
    } else if (type === 'Turnos') {
        text += num + '. ' + (item['Auto'] || item['Nombre y Apellido'] || '-')
+ '\n';
        text += 'Motivo: ' + (item['Motivo'] || '-') + '\n';
        text += 'Fecha: ' + (item['Fecha del Turno'] || '-') + '\n';
        text += 'Horario: ' + (item['Horario'] || '-') + '\n';
        text += 'Estado: ' + (item['Estado'] || '-') + '\n';
    }
    text += '\n';
}
text += '\u00bfQuer\u00e9s avanzar a la siguiente p\u00e1gina o volver a la
anterior?';

const prevBtnData = (totalPages > 1 && page > 1) ? 'paginate:' + type + ':'
+ (page - 1) : 'noop';
const nextBtnData = (totalPages > 1 && page < totalPages) ? 'paginate:' +
type + ':' + (page + 1) : 'noop';

const r = {pageText: text, chatId, messageId, queryId, prevBtnData,
nextBtnData, currentPage: page, totalPages};
return [{json: r}];

```

Anexo 9.4. Modelo Lógico de Datos y Estructura Relacional en Google Sheets

El repositorio centralizado en Google Sheets ("Taller mecánico") está compuesto por tres hojas estructuradas. A continuación se detallan los tipos de datos, restricciones y mecanismos de acceso por herramienta:

Hoja (sheetName)	Nombre del Campo	Clave / Restricción	Visibilidad	Uso en Workflows y Herramientas
stock	Repuesto	Texto	Pública / Admin	Nombre comercial del producto.

stock	Código	Clave Primaria (PK)	Pública / Admin	Clave de búsqueda exacta (filtersUI) en Actualizar Stock1 .
stock	Precio de Costo	Numérico	Solo Admin	Oculto en Consultar_Stock_Publico1 ; accesible en Obtener Repuestos .
stock	Precio Unitario	Numérico	Pública / Admin	Precio de venta final expuesto al cliente.
stock	Stock Actual	Entero	Pública / Admin	Modificable únicamente vía Actualizar Stock1 tras confirmación explícita.
clientes	ID_Cliente	Clave Primaria (PK)	Interna	Calculado incrementalmente como $\text{\$}\max(\text{ID_Cliente}) + 1\$$.
clientes	Nombre y Apellido	Texto	Pública / Admin	Nombre completo registrado en el alta Registrar Cliente1 .
clientes	DNI	Clave Única (UK)	Pública / Admin	Clave de filtro principal en Obtener El ID por DNI y Herramienta_Verificar_DNI1 .
clientes	Número De Teléfono	Texto / Numérico	Pública / Admin	Contacto telefónico opcional del cliente.
clientes	Dirección	Texto	Pública / Admin	Domicilio del cliente.
turnos	ID_Turno	Clave Primaria (PK)	Interna	Calculado incrementalmente como $\text{\$}\max(\text{ID_Turno}) + 1\$$.
turnos	ID_Cliente	Clave Foránea (FK)	Interna	Relación 1:N hacia clientes . Filtro en Obtener Turno Por id .
turnos	Auto	Texto	Pública / Admin	Marca y modelo del vehículo del cliente.

turnos	Motivo	Texto	Pública / Admin	Razón del servicio o diagnóstico preliminar.
turnos	Fecha del Turno	Fecha (dd/MM/yyyy)	Pública / Admin	Evaluated en concurrencia por SubWorkflow_Consultar_Disponibilidad.
turnos	Horario	Hora (HH:mm:ss)	Pública / Admin	Evaluated en concurrencia por SubWorkflow_Consultar_Disponibilidad.
turnos	Estado	Texto (Pendiente, etc.)	Pública / Admin	Controla cupo máximo (3 pendientes) y liberación de agenda al ser Cancelado.

Anexo 9.5. Protocolo y Guion Ampliado de Relevamiento Organizacional

Guion semiestructurado utilizado en la caracterización operativa del caso de estudio, estructurado con preguntas de indagación técnica directa:

Guion semiestructurado utilizado en la caracterización operativa del caso de estudio, estructurado con preguntas de indagación técnica directa:

1. ¿Cuál es el soporte físico o digital exacto utilizado actualmente para registrar las citas (ej. cuaderno de mostrador, planilla de cálculo, calendario de teléfono personal)?
2. ¿Qué volumen promedio diario y semanal de solicitudes de turno se reciben por canales telefónicos o mensajería instantánea (WhatsApp)?
3. ¿Con qué frecuencia mensual ocurren solapamientos de turnos o errores en la asignación de horarios por falta de centralización?
4. ¿Qué porcentaje aproximado de mensajes de clientes solicitando turnos o presupuestos ingresa fuera del horario comercial (18:00 a 08:00 o domingos)?

5. ¿Cómo se procede en la actualidad cuando un cliente necesita reprogramar o cancelar una cita ya agendada?

Bloque 2: Control e Integridad del Inventario de Repuestos

1. ¿Quién es el responsable directo de actualizar la planilla de existencias (stock) al momento de adquirir o utilizar repuestos?
2. ¿Con qué periodicidad real se concilian las existencias físicas del taller contra el registro en Excel?
3. ¿Cuántas veces en el último trimestre se debió detener una reparación en curso por detectar un faltante no registrado en el inventario?
4. ¿Cuánto tiempo en minutos insume habitualmente verificar la disponibilidad y el precio al público de un repuesto durante una consulta telefónica?

Bloque 3: Gestión de Clientes y Comunicación

1. ¿De qué manera se almacena el historial de reparaciones previas realizadas al vehículo de un cliente habitual?
2. Al ingresar un vehículo, ¿se verifica la identidad del propietario mediante documento (DNI) o únicamente por el nombre de pila y modelo del auto?
3. ¿Existe algún mecanismo activo para enviar recordatorios o confirmaciones previas a la fecha del turno agendado?

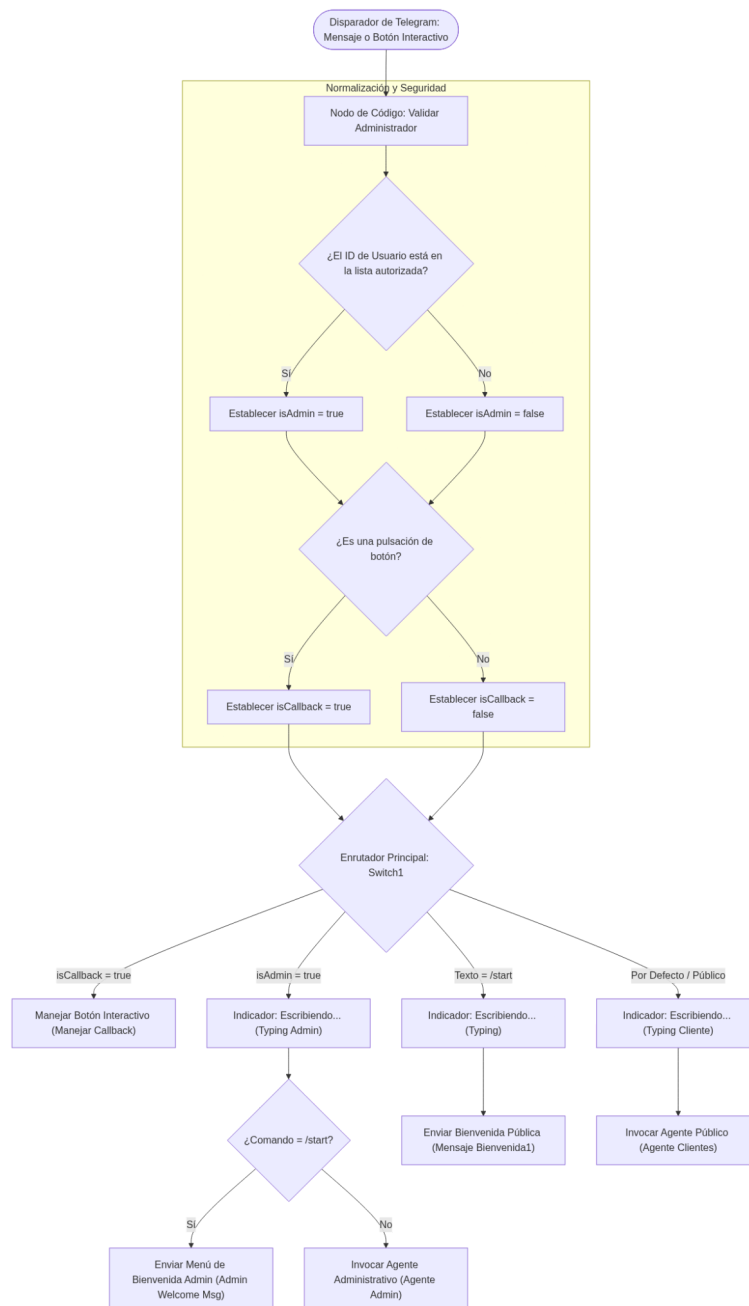
Bloque 4: Barreras de Adopción y Factores Tecnológicos

1. ¿Qué nivel de familiaridad presenta el personal del taller con interfaces de mensajería (Telegram / WhatsApp) versus consolas administrativas de software tradicional?
2. ¿Cuál es el principal factor de rechazo frente a sistemas de gestión comercial (costo de licencias, complejidad de interfaz, tiempo de carga de datos)?
3. ¿Existe disposición operativa para validar las modificaciones de inventario interactuando mediante botones en una interfaz móvil de Telegram?

Anexo 9.6. Diagramas de Flujo de la Arquitectura del Sistema

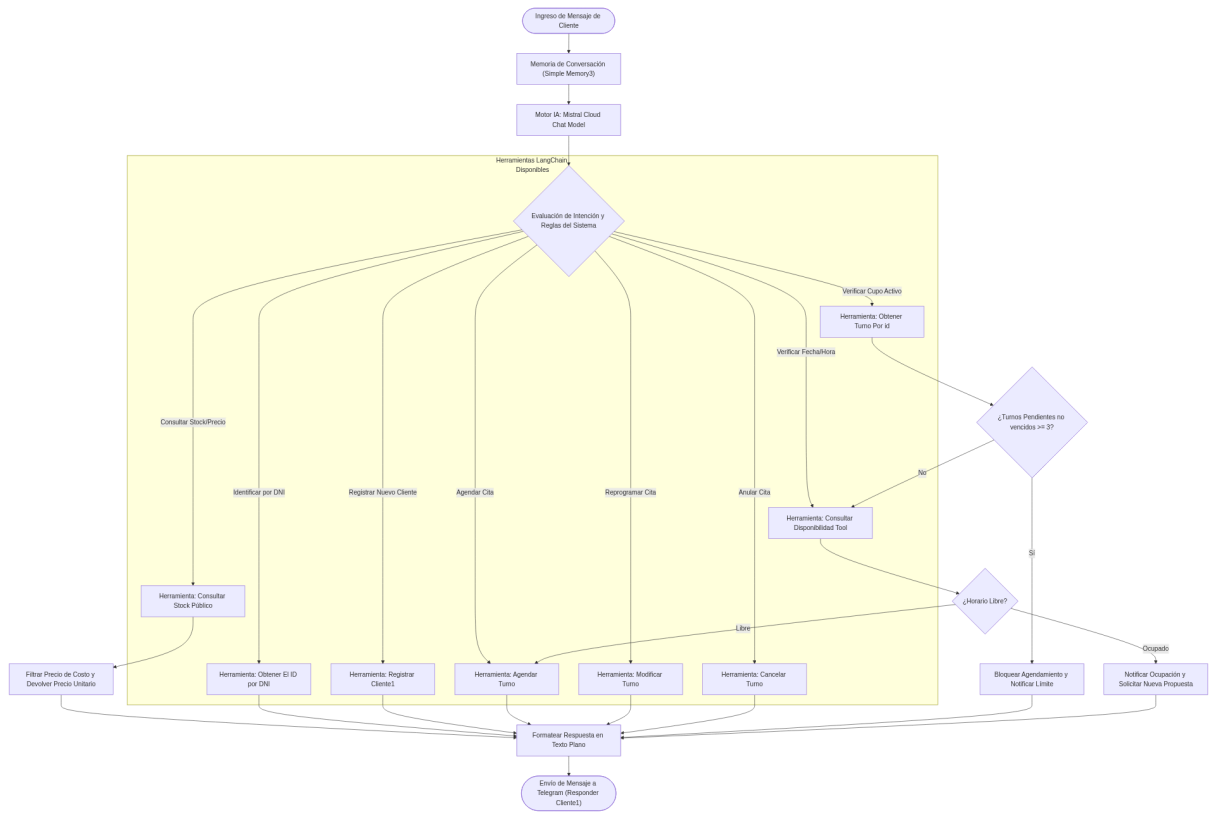
9.6.1. Arquitectura de Enrutamiento y Control de Seguridad (Orquestador Principal)

Representa la capa de entrada, normalización de eventos y validación de privilegios ejecutada antes de invocar los modelos de lenguaje.



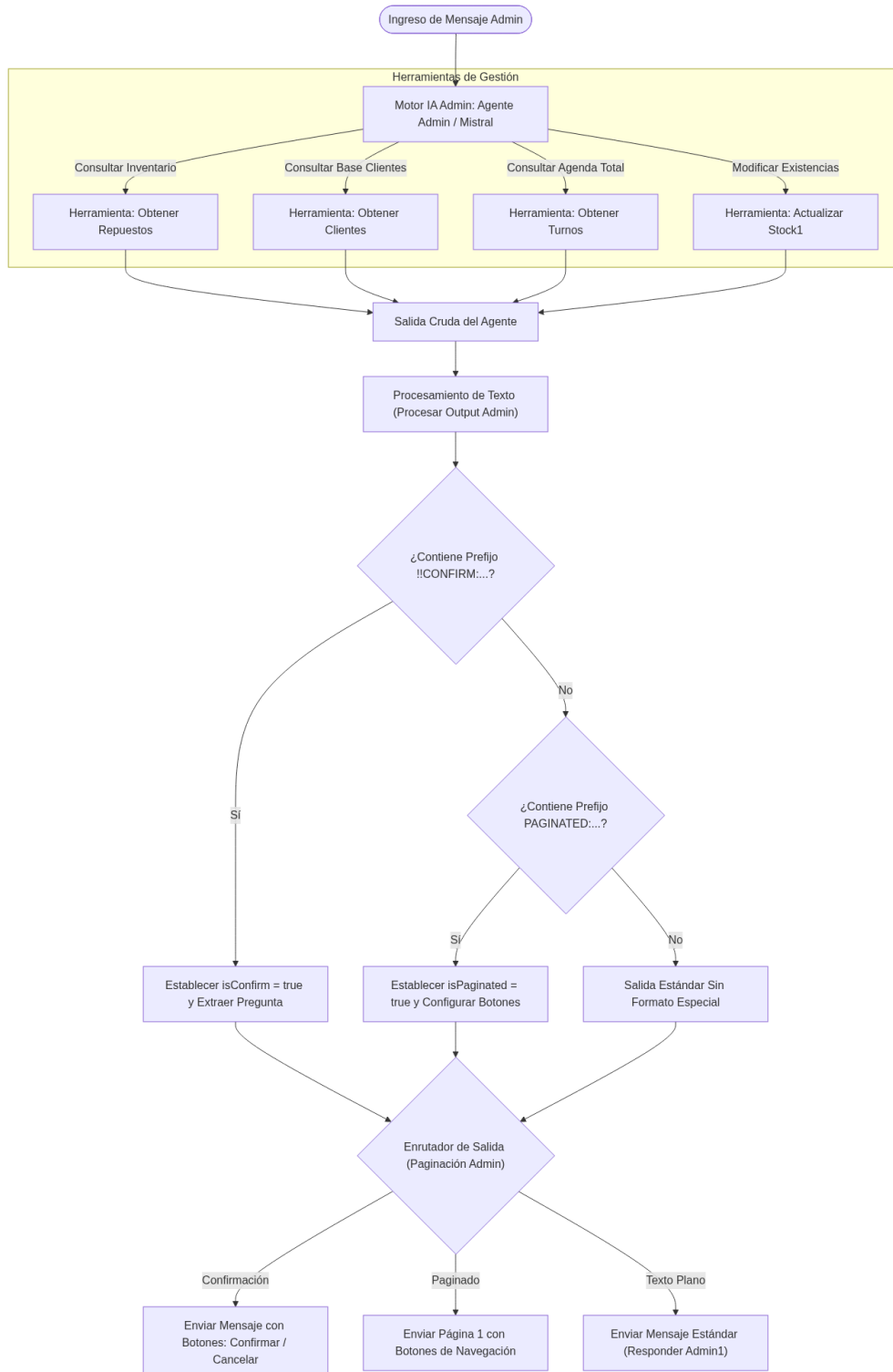
9.6.2. Ciclo de Razonamiento y Herramientas del Agente Público (Servicio Técnico)

Ilustra el flujo conversacional, la invocación de herramientas y la aplicación de reglas de negocio (cupó máximo y verificación de disponibilidad).



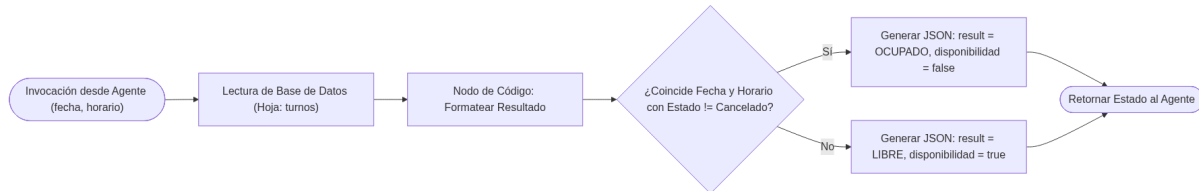
9.6.3. Flujo del Agente Administrativo y Mecanismo de Confirmación

Detalla el procesamiento interno para operaciones administrativas críticas, interceptación de prefijos de control (!!CONFIRM:... / PAGINATED:...) y generación de teclado en línea.



9.6.4. Subworkflow de Verificación de Disponibilidad Horaria

Representación de la lógica booleana ejecutada por el subflujo independiente (SubWorkflow_Consultar_Disponibilidad).



9.6.5. Lógica de Paginación Dinámica y Edición de Mensajes (In-Place)

Muestra el procesamiento de pulsaciones de botones (callbacks) para navegar listados extensos modificando el mensaje original sin generar spam en el chat.

